

Contents

1. Objective	3
2. Reference Components	3
3. Approach	3
4. System Architecture and Workflow	3
4.1 Request workflow for one prediction	4
4.2 Which UI feature uses which API call	4
4.3 Design decisions	4
4.4 Running the integrated application	5
4.5 Issues found by testing and resolved	5
5. Source Code	5
6. Screenshots of the Interface	19
6.1 Connected Frontend and Real-Time Prediction	19
6.2 Request and Response Shown in the UI	20
6.3 Test-Time Augmentation through the Batch Endpoint	21
6.4 Drawing a Digit	22
6.5 Uploading a Photo	23
6.6 Batch Test on Raw Images	24
6.7 Batch Test with Frontend Preprocessing	25
6.8 API Explorer: Handled Errors (400 and 404)	26
6.9 API Explorer: One Bad Image inside a Batch	27
6.10 Request Log	28
6.11 API Offline: Graceful Degradation	29
7. Test Report and Results	30
7.1 Summary	30
7.2 API integration tests	30
7.3 Browser end-to-end checks	31
7.4 Accuracy measured through the HTTP API	32
7.5 Latency, throughput and recovery	32
7.6 Flask server console log during the browser test	33
8. Observations	33
9. Known Limitations	33
10. Deliverables Produced	34
11. Conclusion	34

1. Objective

To establish communication between frontend and backend components for real-time inference: a Streamlit user interface that sends the user's input to a separate Flask REST API, receives the model's prediction as JSON and displays it immediately. The frontend contains no model code; the model lives only behind the API.

2. Reference Components

Flask API — the REST service built in Task 4 (`flask_api/app.py`), used here without any change. It exposes `GET /` , `GET /health` , `POST /predict` (one 8×8 image) and `POST /predict/batch` (up to 100 images), validates every request and answers with JSON, including structured JSON errors (400, 404, 405, 500).

Streamlit UI — new in this task (`frontend/streamlit_app.py`). It lets the user pick a dataset digit, draw one on a canvas or upload a photo, sends it to the API, and shows the predicted digit, confidence, probability chart, round-trip time and the raw request and response. Further tabs run batch tests, send arbitrary requests to the API and show a request log.

Model served. The backend loads `cnn_digits_model.pkl` (fingerprint `94e8348e89`), the frozen from-scratch NumPy CNN also used in the Task 7 application. The Task 1 weights file that the Task 4 report served was not part of the files supplied for this task. The unchanged Task 4 code loads this file without modification and predicts the same digits as Task 4's logged test (0, 3, 7 and 8), but with slightly different confidences (for example 0.9901, 0.9939 and 1.0000 for the 3, 7 and 8 test images, against 0.9998, 0.9827 and 0.9992 in the Task 4 log). The integration does not depend on which weights file is present: copying the Task 1 file into `flask_api/` requires no code change.

3. Approach

1. **Connect the Streamlit frontend with the Flask backend** — the frontend talks to the API only through `frontend/api_client.py` , a small `requests` wrapper. The API address is a sidebar setting (default `http://127.0.0.1:5000` , or the `DIGIT_API_URL` variable), and a health check on every page update shows whether the backend is reachable (Figures 1 and 11).
2. **Transfer user inputs through API requests** — every input mode (sample, drawing, upload) is converted to an 8×8 image, flattened to 64 numbers and sent as JSON to `POST /predict` . Drawings and photos are first cropped, scaled and centred by `frontend/preprocessing.py` so that they resemble the training data (Figures 4 and 5).
3. **Generate predictions in real time** — the reply arrives in a few milliseconds (Section 7.5); the frontend shows the measured round-trip time next to every result (Figure 2). Test-time augmentation sends 11 variants of the image in a single batch request and averages the returned probabilities (Figure 3).
4. **Display outputs dynamically** — predicted digit, confidence band, top-3 ranking, probability chart, batch accuracy table, error messages and a running request log all update from the live JSON replies (Figures 1–10).
5. **Validate the workflow through testing** — an automated suite of 27 API integration tests and a 12-step browser end-to-end test (a real browser driving Streamlit, which calls the real Flask server) check correct predictions, error handling, equality with the local model, accuracy, latency, concurrency and recovery after the API is stopped (Section 7).

4. System Architecture and Workflow

Two independent server processes cooperate. Streamlit serves the user interface and calls the API from the server side; Flask owns the model. They only share the JSON contract below.

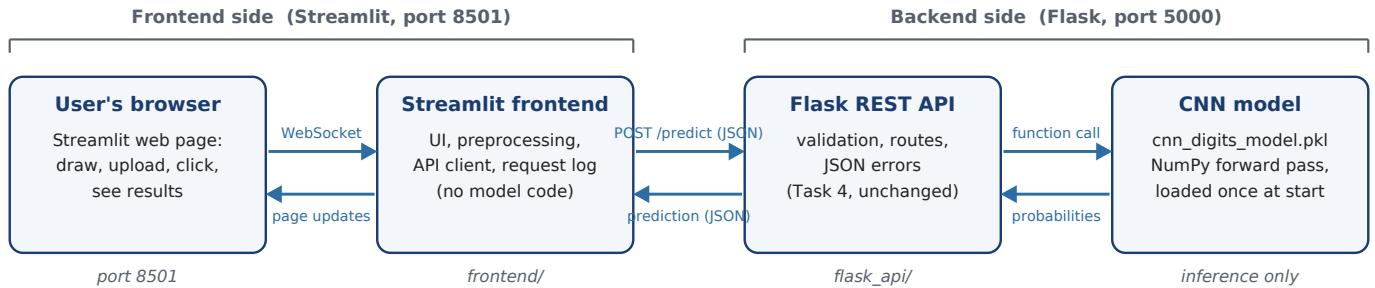


Diagram 1: Components, processes and the messages exchanged between them.

4.1 Request workflow for one prediction

1. The user selects a sample, draws a digit or uploads a photo in the browser.
2. Streamlit converts the input to an 8×8 array in the 0–16 range (cropping, scaling and centring drawings and photos).
3. `DigitAPIClient.predict` sends `POST /predict` with the body `{"image": [64 numbers]}` and starts a timer.
4. Flask validates the body (`parse_image` : valid JSON, exactly 64 numeric values, range 0–16 or 0–1), runs the CNN and returns `predicted_digit` , `confidence` and the ten `probabilities` as JSON.
5. The client stops the timer and returns an `APIResult` object (status, data, latency, or a connection error). It never raises, so the UI can always show something sensible.
6. Streamlit renders the digit card, top-3 list, probability chart, round-trip time, and the raw JSON; the call is added to the request log.

4.2 Which UI feature uses which API call

UI feature	HTTP call	Purpose
Sidebar connection status	<code>GET /health</code>	Checked on every page update; shows online/offline, model loaded and latency; disables Send when offline.
Predict tab (single)	<code>POST /predict</code>	One 8×8 image from a sample, the canvas or an uploaded photo.
Predict tab with test-time augmentation	<code>POST /predict/batch</code>	11 shifted/thicker/thinner variants in one request; probabilities averaged in the frontend.
Batch test tab	<code>POST /predict/batch</code>	Up to 100 dataset images in one request; accuracy, confidence and timing computed from the reply.
API explorer tab	any request	Preset valid and deliberately broken requests (400, 404, 405) sent exactly as typed, including invalid JSON.
Request log tab	none	Every call made in the session with status and latency, downloadable as CSV.

4.3 Design decisions

- **Model only in the backend.** `streamlit_app.py` imports no model code; replacing the weights or the model never touches the frontend.
- **Preprocessing in the frontend.** The API takes an 8×8 image and nothing else. Hand-drawn and photographed digits differ in size and position from the dataset, so the frontend normalises them first. This matters: through the API, the same 270 validation images score 90.74% as raw images and 98.52% after preprocessing (Section 7.4).
- **Test-time augmentation through the batch endpoint.** The 11 variants travel in one request, which keeps the API unchanged and shows the batch endpoint doing real work.

- **Errors are data.** `APIResult` carries either the reply or a readable connection error (offline, timeout). The UI therefore degrades gracefully instead of showing a stack trace.
- **Server-side calls.** Requests are made by the Streamlit process, not by the browser, so the API address must be reachable from the machine running Streamlit and no CORS setup is needed.

4.4 Running the integrated application

1. Install dependencies: `pip install -r requirements.txt` (Streamlit 1.53 or newer).
2. Start the backend in one terminal: `cd flask_api` then `python app.py` (serves on port 5000).
3. Start the frontend in a second terminal: `streamlit run frontend/streamlit_app.py` (opens on `http://localhost:8501`).
4. Run the tests: `python tests/run_integration_tests.py` and `python tests/e2e_ui_tests.py`.

4.5 Issues found by testing and resolved

The end-to-end test found real defects in the first version of the frontend; each was fixed and the full suite re-run until every check passed.

Issue found	Found by	Fix
Drawing a digit crashed the page with a <code>RuntimeError</code> ("image_data was not requested")	UI test U04 (drawing)	Added <code>return_image_data=True</code> to the canvas component.
The raw-request panel listed all 64 pixel values on separate lines, making the page very tall	Screenshot review	Request shown as one wrapped JSON block (<code>st.code</code>).
The endpoint metric was truncated to "POST /pre..."	Screenshot review	Metric now shows only the path (/predict or /predict/batch).
Batch table showed confidence as 59.700000 and a plain checkbox column	Screenshot review	Column formatting set to one decimal; the correct column is a read-only checkbox.
Offline state looked misleading when an older result was still on screen	Screenshot review	The offline test now starts from a fresh visit, as a real user would.

5. Source Code

All application and test code, one file per subsection. Line numbers refer to each file. The Flask API is exactly the Task 4 file. Lines longer than 132 characters are wrapped and continued with "↳"; 21 emoji and symbol characters that the PDF's monospaced font cannot draw are shown as Unicode code points, e.g. `<U+1F522>`.

5.1 flask_api/app.py — Flask API (Task 4, unchanged) (251 lines)

```

1  """
2  Task 4: Developing a Flask API for Deep Learning Models
3  =====
4  Exposes the CNN digit-classification model trained in Task 1 (a from-scratch
5  NumPy CNN, saved as cnn_digits_model.pkl) as a REST API using Flask.
6
7  Endpoints:
8  GET / - API information
9  GET /health - health check
10 POST /predict - predict the digit for a single 8x8 grayscale image
11 POST /predict/batch - predict digits for multiple images in one request
12
13 Run with: python app.py
14 Then send requests to http://127.0.0.1:5000/
15 """
16
17 import os
18 import pickle
19 import traceback
20 import numpy as np
21 from flask import Flask, request, jsonify
22
23 try:
24     BASE_DIR = os.path.dirname(os.path.abspath(__file__))
25 except NameError:
26     # __file__ is not defined when this code runs inside a notebook cell
27     # (Jupyter/Colab) rather than as a standalone script - fall back to the
28     # current working directory in that case.
29     BASE_DIR = os.getcwd()
30 MODEL_PATH = os.path.join(BASE_DIR, "cnn_digits_model.pkl")
31
32 app = Flask(__name__)
33
34 # -----
35 # Model definition (must match the architecture used to train and save
36 # cnn_digits_model.pkl in Task 1) and inference-only forward pass.
37 # -----

```

```

38
39
40 def im2col(x, kh, kw, stride=1, pad=0):
41     N, H, W, C = x.shape
42     if pad > 0:
43         x = np.pad(x, ((0, 0), (pad, pad), (pad, pad), (0, 0)))
44         out_h = (H + 2 * pad - kh) // stride + 1
45         out_w = (W + 2 * pad - kw) // stride + 1
46         cols = np.zeros((N, out_h, out_w, kh, kw, C), dtype=x.dtype)
47         for i in range(kh):
48             i_max = i + stride * out_h
49             for j in range(kw):
50                 j_max = j + stride * out_w
51                 cols[:, :, :, i, j, :] = x[:, i:i_max:stride, j:j_max:stride, :]
52         return cols.reshape(N, out_h, out_w, kh * kw * C), out_h, out_w
53
54
55 def conv_forward(x, W, b, stride=1, pad=1):
56     N, H, Wd, C = x.shape
57     k = W.shape[0]
58     out_ch = W.shape[-1]
59     cols, out_h, out_w = im2col(x, k, k, stride, pad)
60     W_col = W.reshape(-1, out_ch)
61     out = cols.reshape(N * out_h * out_w, -1) @ W_col + b
62     return out.reshape(N, out_h, out_w, out_ch)
63
64
65 def relu(x):
66     return np.maximum(0, x)
67
68
69 def maxpool_forward(x, size=2, stride=2):
70     N, H, W, C = x.shape
71     out_h, out_w = H // stride, W // stride
72     x = x[:, :out_h * stride, :out_w * stride, :]
73     out = np.zeros((N, out_h, out_w, C), dtype=x.dtype)
74     for i in range(out_h):
75         for j in range(out_w):
76             window = x[:, i*stride:i*stride+size, j*stride:j*stride+size, :]
77             out[:, i, j, :] = window.max(axis=(1, 2))
78     return out
79
80
81 def softmax(logits):
82     e = np.exp(logits - logits.max(axis=1, keepdims=True))
83     return e / e.sum(axis=1, keepdims=True)
84
85
86 class DigitCNN:
87     """Loads the Task 1 CNN weights and runs inference only (no training)."""
88
89     def __init__(self, weights_path):
90         with open(weights_path, "rb") as f:
91             self.w = pickle.load(f)
92
93     def predict_proba(self, x):
94         """x: numpy array of shape (N, 8, 8, 1), pixel values already in [0, 1]."""
95         w = self.w
96         x = conv_forward(x, w["conv1_W"], w["conv1_b"], stride=1, pad=1)
97         x = relu(x)
98         x = maxpool_forward(x, 2, 2)
99         x = conv_forward(x, w["conv2_W"], w["conv2_b"], stride=1, pad=1)
100        x = relu(x)
101        x = maxpool_forward(x, 2, 2)
102        x = x.reshape(x.shape[0], -1)
103        x = x @ w["fc1_W"] + w["fc1_b"]
104        x = relu(x)
105        logits = x @ w["fc2_W"] + w["fc2_b"]
106        return softmax(logits)
107
108
109 # Load the model once at startup, not on every request.
110 model = DigitCNN(MODEL_PATH)
111
112
113 # -----
114 # Helpers
115 # -----
116
117 def parse_image(payload):
118     """
119     Validates and converts a single image payload into a (1, 8, 8, 1)
120     float32 array normalized to [0, 1]. Raises ValueError with a clear
121     message on any problem, which the route handlers turn into a 400.
122     """
123     if payload is None:
124         raise ValueError("Missing 'image' field in request body.")
125
126     arr = np.array(payload, dtype=np.float64)
127
128     if arr.size != 64:
129         raise ValueError(
130             f"Expected an 8x8 (64-value) grayscale image, got {arr.size} values."
131         )
132
133     arr = arr.reshape(8, 8)
134
135     if np.isnan(arr).any():
136         raise ValueError("Image contains non-numeric or missing values.")
137
138     # Accept either raw 0-16 pixel scale (like the original digits dataset)
139     # or already-normalized 0-1 scale, and normalize consistently to [0, 1].
140     if arr.max() > 1.0:
141         if arr.max() > 16.0 or arr.min() < 0.0:
142             raise ValueError("Pixel values must be within [0, 16] (or already normalized to [0, 1]).")
143         arr = arr / 16.0
144     elif arr.min() < 0.0:
145         raise ValueError("Pixel values must not be negative.")
146
147     return arr.reshape(1, 8, 8, 1).astype(np.float32)
148

```

```

149
150 def format_prediction(probs_row):
151     probs_row = probs_row.tolist()
152     pred_digit = int(np.argmax(probs_row))
153     return {
154         "predicted_digit": pred_digit,
155         "confidence": round(probs_row[pred_digit], 4),
156         "probabilities": {str(i): round(p, 4) for i, p in enumerate(probs_row)},
157     }
158
159 # -----
160 # Routes
161 # -----
162
163
164 @app.route("/", methods=["GET"])
165 def index():
166     return jsonify({
167         "name": "Digit Classification API",
168         "description": "REST API serving the Task 1 from-scratch CNN digit classifier.",
169         "endpoints": {
170             "GET /health": "Health check.",
171             "POST /predict": "Predict the digit for one 8x8 grayscale image. "
172                 "Body: {\\"image\\": [64 numbers, 0-16 or 0-1, row-major 8x8]}",
173             "POST /predict/batch": "Predict digits for multiple images. "
174                 "Body: {\\"images\\": [[64 numbers], [64 numbers], ...]}",
175         },
176     })
177
178
179 @app.route("/health", methods=["GET"])
180 def health():
181     return jsonify({"status": "ok", "model_loaded": model is not None})
182
183
184 @app.route("/predict", methods=["POST"])
185 def predict():
186     try:
187         data = request.get_json(silent=True)
188         if data is None:
189             return jsonify({"error": "Request body must be valid JSON."}), 400
190
191         image = parse_image(data.get("image"))
192         probs = model.predict_proba(image)[0]
193         return jsonify(format_prediction(probs)), 200
194
195     except ValueError as e:
196         return jsonify({"error": str(e)}), 400
197     except Exception as e:
198         app.logger.error("Unexpected error in /predict: %s\\n%s", e, traceback.format_exc())
199         return jsonify({"error": "Internal server error while generating prediction."}), 500
200
201
202 @app.route("/predict/batch", methods=["POST"])
203 def predict_batch():
204     try:
205         data = request.get_json(silent=True)
206         if data is None:
207             return jsonify({"error": "Request body must be valid JSON."}), 400
208
209         images_payload = data.get("images")
210         if not isinstance(images_payload, list) or len(images_payload) == 0:
211             return jsonify({"error": "'images' must be a non-empty list of 64-value images."}), 400
212         if len(images_payload) > 100:
213             return jsonify({"error": "Batch size limited to 100 images per request."}), 400
214
215         results = []
216         for idx, img_payload in enumerate(images_payload):
217             try:
218                 image = parse_image(img_payload)
219                 probs = model.predict_proba(image)[0]
220                 results.append(format_prediction(probs))
221             except ValueError as e:
222                 results.append({"error": f"image[{idx}]: {e}"})
223
224         return jsonify({"count": len(results), "results": results}), 200
225
226     except Exception as e:
227         app.logger.error("Unexpected error in /predict/batch: %s\\n%s", e, traceback.format_exc())
228         return jsonify({"error": "Internal server error while generating predictions."}), 500
229
230
231 # -----
232 # Error handlers for common HTTP errors
233 # -----
234
235 @app.errorhandler(404)
236 def not_found(e):
237     return jsonify({"error": "Endpoint not found. See GET / for a list of available endpoints."}), 404
238
239
240 @app.errorhandler(405)
241 def method_not_allowed(e):
242     return jsonify({"error": "Method not allowed for this endpoint."}), 405
243
244
245 @app.errorhandler(500)
246 def server_error(e):
247     return jsonify({"error": "Internal server error."}), 500
248
249
250 if __name__ == "__main__":
251     app.run(host="0.0.0.0", port=5000, debug=False)

```

5.2 frontend/api_client.py — HTTP client with error handling (116 lines)

```
1 """
2 api_client.py
3 =====
4 HTTP client used by the Streamlit frontend to talk to the Flask API (Task 4).
5
6 The frontend never loads the model: every prediction is a JSON request to the
7 Flask backend. This module wraps those requests so that
8 * every call returns an APIResult (never raises) - connection problems,
9   timeouts and HTTP errors are all reported as data the UI can display;
10 * the round-trip latency of every call is measured;
11 * every call can be recorded in the UI's request log through a callback.
12 """
13 import json
14 import time
15 from dataclasses import dataclass
16 from typing import Any, Callable, Optional
17
18 import requests
19
20
21 @dataclass
22 class APIResult:
23     ok: bool # True only for a 2xx response
24     status: Optional[int] # HTTP status, or None if the server was unreachable
25     data: Any # parsed JSON body (dict) or None
26     latency_ms: float # full round-trip time measured by the client
27     method: str
28     path: str
29     request_body: Any = None # what was sent (for display in the UI)
30     error: Optional[str] = None # connection-level problem (offline / timeout / ...)
31
32     @property
33     def unreachable(self) -> bool:
34         return self.status is None
35
36     @property
37     def message(self) -> str:
38         """Human-readable one-line summary of a failed call."""
39         if self.unreachable:
40             return self.error or "Cannot reach the API."
41         if isinstance(self.data, dict) and "error" in self.data:
42             return f"HTTP {self.status}: {self.data['error']}"
43         return f"HTTP {self.status}"
44
45
46 class DigitAPIClient:
47     def __init__(self, base_url: str = "http://127.0.0.1:5000", timeout: float = 5.0,
48                  on_call: Optional[Callable[[APIResult], None]] = None):
49         self.base_url = base_url.rstrip("/")
50         self.timeout = timeout
51         self.on_call = on_call
52         self.session = requests.Session()
53
54     # -----
55     def _call(self, method: str, path: str, json_body: Any = None, raw_body: Optional[str] = None,
56              record: bool = True) -> APIResult:
57         url = self.base_url + path
58         sent = json_body if raw_body is None else raw_body
59         t0 = time.perf_counter()
60         try:
61             if raw_body is not None: # used by the API explorer to send deliberately broken JSON
62                 resp = self.session.request(method, url, data=raw_body.encode("utf-8"),
63                                             headers={"Content-Type": "application/json"}, timeout=self.timeout)
64             else:
65                 resp = self.session.request(method, url, json=json_body, timeout=self.timeout)
66             ms = (time.perf_counter() - t0) * 1000
67             try:
68                 data = resp.json()
69             except ValueError:
70                 data = {"error": "Server returned a non-JSON response.", "text": resp.text[:200]}
71             result = APIResult(resp.ok, resp.status_code, data, ms, method, path, sent)
72             except requests.exceptions.ConnectionError:
73                 result = APIResult(False, None, None, (time.perf_counter() - t0) * 1000, method, path, sent,
74                                   error=f"Cannot reach the Flask API at {self.base_url} (connection refused).")
75             except requests.exceptions.Timeout:
76                 result = APIResult(False, None, None, (time.perf_counter() - t0) * 1000, method, path, sent,
77                                   error=f"The Flask API did not answer within {self.timeout:g} s (timeout).")
78             except requests.exceptions.RequestException as e:
79                 result = APIResult(False, None, None, (time.perf_counter() - t0) * 1000, method, path, sent,
80                                   error=f"Request failed: {e}")
81             if record and self.on_call:
82                 self.on_call(result)
83             return result
84
85     # ----- endpoints
86     def info(self) -> APIResult:
87         return self._call("GET", "/")
88
89     def health(self, record: bool = False) -> APIResult:
90         return self._call("GET", "/health", record=record)
91
92     @staticmethod
93     def _payload(pixels) -> list:
94         """8x8 array -> flat list of 64 numbers in the 0-16 range (the API's native scale)."""
95         flat = [min(16.0, max(0.0, round(float(v), 4))) for v in list(_flatten(pixels))]
96         return flat
97
98     def predict(self, pixels) -> APIResult:
99         return self._call("POST", "/predict", {"image": self._payload(pixels)})
100
101     def predict_batch(self, images) -> APIResult:
102         return self._call("POST", "/predict/batch", {"images": [self._payload(p) for p in images]})
103
104     def custom(self, method: str, path: str, body_text: Optional[str] = None) -> APIResult:
105         """Free-form request used by the API explorer (body is sent exactly as typed)."""
106         if method == "GET" or not body_text:
107             return self._call(method, path)
108         return self._call(method, path, raw_body=body_text)
```

```

109
110
111 def _flatten(x):
112     try:
113         import numpy as np
114         return np.asarray(x, dtype=float).reshape(-1).tolist()
115     except Exception:
116         return [v for row in x for v in row]

```

5.3 frontend/preprocessing.py — Image preprocessing and test-time-augmentation variants (92 lines)

```

1  """
2  preprocessing.py
3  =====
4  Frontend-side image preparation (copied unchanged from the Task 7 application).
5
6  Hand-drawn or photographed digits are arbitrary sizes and positions, while the
7  Flask API (and the CNN behind it) expects an 8x8 image in the same style as the
8  scikit-learn Digits dataset. These helpers crop to the ink, scale, centre by
9  mass and rescale to 0-16 *before* the image is sent to the API, so the API
10 itself stays a thin, model-only service.
11
12 tta_variants() builds the 11 slightly shifted / thickened / thinned copies of an
13 image used for test-time augmentation: the frontend sends them to the API in ONE
14 POST /predict/batch call and averages the returned probabilities.
15 """
16 import numpy as np
17 from PIL import Image
18
19
20 def pixels_to_display_image(p, size=192):
21     arr = np.clip(p, 0, 16) / 16.0 * 255
22     return Image.fromarray(arr.astype(np.uint8)).resize((size, size), Image.NEAREST)
23
24
25 def crop_to_ink(gray, pad_frac=0.07):
26     arr = gray.astype(np.float64)
27     if arr.max() <= arr.min():
28         return None
29     norm = (arr - arr.min()) / (arr.max() - arr.min()) * 255.0
30     mask = norm > max(25.0, norm.max() * 0.18)
31     if not mask.any():
32         return None
33     ys, xs = np.where(mask)
34     y0, y1, x0, x1 = ys.min(), ys.max(), xs.min(), xs.max()
35     py, px = max(1, int((y1 - y0 + 1) * pad_frac)), max(1, int((x1 - x0 + 1) * pad_frac))
36     y0, y1 = max(0, y0 - py), min(norm.shape[0] - 1, y1 + py)
37     x0, x1 = max(0, x0 - px), min(norm.shape[1] - 1, x1 + px)
38     return norm[y0:y1 + 1, x0:x1 + 1]
39
40
41 def shift_2d(arr, sy, sx):
42     out = np.zeros_like(arr)
43     h, w = arr.shape
44     out[max(0, sy):min(h, h + sy), max(0, sx):min(w, w + sx)] = \
45         arr[max(0, -sy):min(h, h - sy), max(0, -sx):min(w, w - sx)]
46     return out
47
48
49 def center_by_mass(c):
50     total = c.sum()
51     if total <= 0:
52         return c
53     yy, xx = np.mgrid[0:8, 0:8]
54     cy, cx = (yy * c).sum() / total, (xx * c).sum() / total
55     return shift_2d(c, int(round(3.5 - cy)), int(round(3.5 - cx)))
56
57
58 def dilate3(a):
59     return np.maximum.reduce([shift_2d(a, dy, dx) for dy in (-1, 0, 1) for dx in (-1, 0, 1)])
60
61
62 def erode3(a):
63     return np.minimum.reduce([shift_2d(a, dy, dx) for dy in (-1, 0, 1) for dx in (-1, 0, 1)])
64
65
66 def smart_preprocess_to_8x8(gray):
67     cropped = crop_to_ink(gray)
68     if cropped is None:
69         return np.zeros((8, 8))
70     ch, cw = cropped.shape
71     scale = 7.0 / max(ch, cw)
72     nh = int(np.clip(int(round(ch * scale)), 3.2, 8))
73     nw = int(np.clip(int(round(cw * scale)), 3.2, 8))
74     resized = np.array(Image.fromarray(cropped.astype(np.uint8)).resize((nw, nh), Image.LANCZOS)).astype(np.float64)
75     canvas = np.zeros((8, 8))
76     oy, ox = (8 - nh) // 2, (8 - nw) // 2
77     canvas[oy:oy + nh, ox:ox + nw] = resized
78     canvas = center_by_mass(canvas)
79     return canvas / canvas.max() * 16.0 if canvas.max() > 0 else canvas
80
81
82 def tta_variants(pixels_8x8):
83     """11 variants: the original, 8 one-pixel shifts, one thicker and one thinner copy."""
84     base = np.clip(pixels_8x8, 0, 16)
85     variants = [base]
86     for dy in (-1, 0, 1):
87         for dx in (-1, 0, 1):
88             if dy or dx:
89                 variants.append(shift_2d(base, dy, dx))
90     variants.append(np.clip(dilate3(base), 0, 16))
91     variants.append(np.clip(erode3(base), 0, 16))
92     return variants

```

5.4 frontend/streamlit_app.py — Streamlit frontend (365 lines)

```
1 """
2 streamlit_app.py
3 =====
4 Task 8: Integrating a Streamlit frontend with the Flask API (Task 4).
5
6 The frontend contains NO model code. Every prediction is made by sending a JSON
7 request to the Flask backend over HTTP and displaying the JSON that comes back:
8
9 browser <-> Streamlit (this file) --HTTP/JSON--> Flask API --> CNN model
10
11 Run the backend first (cd flask_api && python app.py), then:
12 streamlit run frontend/streamlit_app.py
13 The API address can be changed in the sidebar or with the DIGIT_API_URL variable.
14 """
15 import json
16 import os
17 import time
18 from datetime import datetime
19
20 import altair as alt
21 import numpy as np
22 import pandas as pd
23 import streamlit as st
24 from PIL import Image
25 from sklearn.datasets import load_digits
26
27 try:
28     from streamlit_drawable_canvas import st_canvas
29     CANVAS_OK = True
30 except Exception:
31     CANVAS_OK = False
32     # component missing or incompatible
33
34 from api_client import DigitAPIClient
35 from preprocessing import crop_to_ink, pixels_to_display_image, smart_preprocess_to_8x8, tta_variants
36
37 DEFAULT_URL = os.environ.get("DIGIT_API_URL", "http://127.0.0.1:5000")
38 DIGITS = [str(i) for i in range(10)]
39
40 st.set_page_config(page_title="Digit Classifier · Streamlit + Flask", page_icon="<U+1F522>", layout="wide")
41
42 st.markdown("""
43 <style>
44 .block-container { padding-top: 4.5rem; padding-bottom: 3rem; max-width: 1250px; }
45 .hero { background: linear-gradient(180deg,#17181c 0%,#101114 100%); border: 1px solid rgba(255,255,255,.08);
46     border-top: 3px solid #FF4B4B; border-radius: 10px; padding: 1.4rem 2rem; margin-bottom: 1.2rem; }
47 .hero .eyebrow { font-family: monospace; font-size: .75rem; letter-spacing: .14em; text-transform: uppercase; color: #9CA3AF; }
48 .hero h1 { margin: .5rem 0 0 0 !important; color: #F5F5F7; font-size: 2rem !important; font-weight: 800 !important; }
49 .hero p { margin: .5rem 0 0 0; color: #c9ccd1; max-width: 820px; line-height: 1.55; }
50 .result-card { border-radius: 18px; padding: 1.4rem 1.2rem; text-align: center; border: 2px solid var(--b); background: var(--bg); }
51 .result-digit { font-size: 4.2rem; font-weight: 800; line-height: 1; margin: .2rem 0; }
52 .result-label { font-size: .9rem; opacity: .75; text-transform: uppercase; letter-spacing: .06em; }
53 .pill { display: inline-block; padding: .3rem .9rem; border-radius: 999px; font-weight: 700; margin-top: .5rem; }
54 .step { display: flex; align-items: center; gap: .6rem; margin-bottom: .45rem; }
55 .badge { width: 28px; height: 28px; border-radius: 50%; background: #FF4B4B; color: #fff; font-weight: 800;
56     display: inline-flex; align-items: center; justify-content: center; flex-shrink: 0; }
57 .footer { text-align: center; opacity: .55; font-size: .85rem; margin-top: 2rem; }
58 </style>
59 """, unsafe_allow_html=True)
60
61 # -----
62 # Request log + API client (one client per rerun, calls are recorded in session state)
63 # -----
64 st.session_state.setdefault("api_log", [])
65
66 def record_call(r):
67     st.session_state.api_log.append({
68         "time": datetime.now().strftime("%H:%M:%S"), "method": r.method, "endpoint": r.path,
69         "status": r.status if r.status is not None else "no response",
70         "latency_ms": round(r.latency_ms, 1), "result": "OK" if r.ok else r.message,
71     })
72
73 @st.cache_data
74 def load_dataset():
75     d = load_digits()
76     return d.images, d.target
77
78 images, labels = load_dataset()
79
80 with st.sidebar:
81     st.markdown("### <U+1F50C> Flask API connection")
82     api_url = st.text_input("API base URL", DEFAULT_URL)
83     timeout = st.slider("Request timeout (s)", 1, 15, 5)
84     st.button("Re-check connection", width="stretch")
85     client = DigitAPIClient(api_url, timeout, on_call=record_call)
86     health = client.health(record=False)
87     API_ONLINE = bool(health.ok and isinstance(health.data, dict) and health.data.get("model_loaded"))
88     if API_ONLINE:
89         st.success(f"***API online** ✓\n\nmodel loaded · {health.latency_ms:.0f} ms")
90     else:
91         st.error(f"***API offline** ✗\n\n{health.message}")
92         st.code("cd flask_api\npython app.py", language="bash")
93     st.divider()
94     log = st.session_state.api_log
95     st.caption(f"Requests sent this session: **{len(log)}**")
96     if log:
97         ok = sum(1 for e in log if e["result"] == "OK")
98         st.caption(f"Successful: **{ok}** · mean latency: **{np.mean([e['latency_ms'] for e in log]):.1f} ms**")
99
100 st.markdown("""
101 <div class="hero"><div class="eyebrow">Task 8 · Streamlit frontend + Flask backend</div>
102 <h1>Digit Classifier</h1>
103 <p>Draw, upload or pick a digit. The Streamlit frontend sends it as JSON to a separate Flask API,
104 which runs the CNN and returns the prediction in real time.</p></div>
105 """, unsafe_allow_html=True)
```

```

109
110
111 def steps(*items):
112     st.markdown("".join(f'<div class="step"><span class="badge">{i}</span><span>{t}</span></div>'
113         for i, t in enumerate(items, 1)), unsafe_allow_html=True)
114
115
116 def confidence_style(c):
117     if c >= 0.8:
118         return "#21C55D", "High confidence"
119     if c >= 0.5:
120         return "#F59E0B", "Moderate confidence"
121     return "#EF4444", "Low confidence"
122
123
124 # -----
125 # Sending a prediction through the API
126 # -----
127 def predict_via_api(pixels, use_tta):
128     """Returns a dict describing the outcome, or {'error': ...} if the API call failed."""
129     if use_tta:
130         variants = tta_variants(pixels)
131         r = client.predict_batch(variants) # one POST /predict/batch with 11 images
132         if not r.ok:
133             return {"error": r.message, "result": r}
134         rows = [x for x in r.data["results"] if "error" not in x]
135         if not rows:
136             return {"error": "The API rejected every variant.", "result": r}
137         probs = np.mean([x["probabilities"][d] for d in DIGITS] for x in rows, axis=0)
138         mode = f"POST /predict/batch ({len(variants)} test-time-augmentation variants averaged)"
139     else:
140         r = client.predict(pixels) # one POST /predict
141         if not r.ok:
142             return {"error": r.message, "result": r}
143         probs = np.array([r.data["probabilities"][d] for d in DIGITS])
144         mode = "POST /predict"
145     pred = int(np.argmax(probs))
146     return {"pred": pred, "conf": float(probs[pred]), "probs": probs, "mode": mode, "result": r, "pixels": pixels}
147
148
149 def render_result(out, true_label=None):
150     if "error" in out:
151         st.error(f"The API call failed. {out['error']}")
152         return
153     r, probs, pred, conf = out["result"], out["probs"], out["pred"], out["conf"]
154     color, word = confidence_style(conf)
155     st.markdown("#### Result from the Flask API")
156     c1, c2, c3 = st.columns([1, 1.1, 1.3])
157     with c1:
158         st.image(pixels_to_display_image(out["pixels"], 220), caption="8x8 image sent to the API", width="stretch")
159     with c2:
160         verdict = ""
161         if true_label is not None:
162             verdict = f'<div style="margin-top: 5rem">{"<U+2705> correct" if pred == true_label else "<U+274C> wrong"} (true label {
163         true_label})</div>'
164         st.markdown(
165             f'<div class="result-card" style="--b:{color}55;--bg:{color}14"><div class="result-label">Predicted digit</div>'
166             f'<div class="result-digit" style="color:{color}>{pred}</div>'
167             f'<span class="pill" style="background:{color}22;color:{color}>{conf*100:.1f}% · {word}</span>{verdict}</div>',
168             unsafe_allow_html=True)
169     with c3:
170         st.markdown("***Top 3 guesses**")
171         for rank, d in enumerate(np.argsort(probs)[-1:][3]):
172             st.markdown(f'["<U+1F947>', '<U+1F948>', '<U+1F949>'][rank]] **{d}** - {probs[d]*100:.1f}%')
173             st.progress(float(min(max(probs[d], 0), 1)))
174     m1, m2, m3, m4 = st.columns(4)
175     m1.metric("Round-trip latency", f"{r.latency_ms:.1f} ms")
176     m2.metric("HTTP status", r.status)
177     m3.metric("Endpoint", out["mode"].split(" ")[0].replace("POST ", ""))
178     m4.metric("Payload sent", f"{len(json.dumps(r.request_body))/1024:.1f} KB")
179     df = pd.DataFrame({"Digit": DIGITS, "Probability": probs, "Top": [int(i == pred for i in range(10))])
180     st.altair_chart(
181         alt.Chart(df).mark_bar(cornerRadiusTopLeft=4, cornerRadiusTopRight=4).encode(
182             x=alt.X("Digit:N", sort=None), y=alt.Y("Probability:Q", scale=alt.Scale(domain=[0, 1])),
183             color=alt.condition(alt.datum.Top, alt.value("#FF4B4B"), alt.value("#5B7FDE88")),
184             tooltip=[f"Digit", alt.Tooltip("Probability:Q", format=".1%")].properties(height=220),
185             width="stretch")
186     with st.expander("Show the raw request and response (JSON)"):
187         a, b = st.columns(2)
188         body = r.request_body
189         shown = body if len(json.dumps(body)) < 900 else {k: (f"[{len(v)} images × 64 values]" if k == "images" else v) for k, v in
190         body.items()}
191         a.markdown(f"***Request** - {out['mode']}")
192         a.code(json.dumps(shown), language="json", wrap_lines=True)
193         resp = r.data
194         if isinstance(resp, dict) and "results" in resp:
195             resp = {"count": resp["count"], "results": f"[{len(resp['results'])} predictions] first: {resp['results'][0]}"}
196         b.markdown(f"***Response** - HTTP {r.status}")
197         b.json(resp)
198
199 # -----
200 tab_predict, tab_batch, tab_explorer, tab_log = st.tabs(
201     ["<U+1F9EA> Predict", "<U+1F4E6> Batch test", "<U+1F6E0><U+FE0F> API explorer", "<U+1F4DC> Request log"])
202
203 # ===== Predict =====
204 with tab_predict:
205     steps("Choose how to provide a digit.", "Press <b>Send to Flask API</b> - the frontend posts JSON and shows the live reply.")
206     mode = st.radio("Input", ["Sample digit", "Draw", "Upload photo"], horizontal=True, label_visibility="collapsed")
207     use_tta = st.checkbox("Test-time augmentation (sends 11 variants in one /predict/batch call and averages them)")
208     pixels, true_label = None, None
209
210     if mode == "Sample digit":
211         fcol, scol, icol = st.columns([1, 2, 1])
212         dfilter = fcol.selectbox("Digit", ["All"] + DIGITS)
213         cand = np.arange(len(images)) if dfilter == "All" else np.where(labels == int(dfilter))[0]
214         pick = scol.slider("Sample number", 1, len(cand), 1)
215         idx = int(cand[pick - 1])
216         pre = icol.checkbox("Crop → scale → center first", value=False)
217         true_label = int(labels[idx])
218         raw = images[idx].astype(float)

```

```

218     pixels = smart_preprocess_to_8x8(np.clip(raw, 0, 16) / 16 * 255) if pre else raw
219     a, b = st.columns([1, 4])
220     a.image(pixels_to_display_image(pixels, 190), width="stretch")
221     b.markdown(f"Dataset index **{idx}** · true label **{true_label}**")
222
223 elif mode == "Draw":
224     if not CANVAS_OK:
225         st.warning("The drawing canvas component is not available here. Use a sample or upload a photo.")
226     else:
227         left, right = st.columns([3, 2])
228         with left:
229             brush = st.slider("Brush size", 8, 40, 22)
230             canvas = st_canvas(fill_color="white", stroke_width=brush, stroke_color="white", background_color="black",
231                               height=360, width=360, drawing_mode="freedraw", update_streamlit=True, return_image_data=True, ke
232 l, y="canvas")
233         if canvas.image_data is not None and canvas.image_data[:, :, :3].sum() > 0:
234             gray = np.array(Image.fromarray(canvas.image_data.astype(np.uint8), mode="RGBA").convert("L"))
235             pixels = smart_preprocess_to_8x8(gray)
236         with right:
237             st.markdown("#### 8x8 image that will be sent")
238             if pixels is not None and pixels.sum() > 0:
239                 st.image(pixels_to_display_image(pixels, 280), width="stretch")
240             else:
241                 pixels = None
242                 st.info("Draw a digit to see the live preview.")
243
244 else:
245     up, opt = st.columns([2, 1])
246     invert = opt.checkbox("Invert colors", value=True, help="ON for a dark digit on a light background.")
247     file = up.file_uploader("Choose an image", type=["png", "jpg", "jpeg"])
248     if file is not None:
249         gray = np.array(Image.open(file).convert("L")).astype(float)
250         gray = 255 - gray if invert else gray
251         crop = crop_to_ink(gray)
252         pixels = smart_preprocess_to_8x8(gray)
253         c1, c2, c3 = st.columns(3)
254         c1.markdown("**Original**"); c1.image(file, width="stretch")
255         c2.markdown("**Auto-cropped**")
256         if crop is not None:
257             c2.image(Image.fromarray(crop.astype(np.uint8)).resize((240, 240), Image.NEAREST), width="stretch")
258             c3.markdown("**8x8 image to send**"); c3.image(pixels_to_display_image(pixels, 240), width="stretch")
259         if pixels.sum() == 0:
260             pixels = None
261             st.warning("No digit found. Try the invert toggle or a clearer photo.")
262
263 send = st.button("<U+1F680> Send to Flask API", type="primary", disabled=(pixels is None or not API_ONLINE))
264 if not API_ONLINE:
265     st.warning("The Flask API is offline, so predictions are unavailable. Start it, then press *Re-check connection*.")
266 if send and pixels is not None:
267     with st.spinner("Waiting for the Flask API..."):
268         st.session_state.last = (predict_via_api(pixels, use_tta), true_label)
269 if st.session_state.get("last"):
270     st.divider()
271     render_result(*st.session_state.last)
272
273 # ===== Batch =====
274 with tab_batch:
275     st.markdown("Send many dataset digits in **one** `POST /predict/batch` request and compare the API's answers with the true label")
276     l, s, )
277     c1, c2, c3, c4 = st.columns(4)
278     n = c1.slider("Images in the batch", 5, 100, 50)
279     src = c2.selectbox("Digits", ["All"] + DIGITS, key="bsrc")
280     seed = c3.number_input("Random seed", 0, 9999, 1)
281     pre_b = c4.checkbox("Crop + scale + center first", value=False, key="bpre")
282     if st.button("> Run batch", type="primary", disabled=not API_ONLINE):
283         pool = np.arange(len(images)) if src == "All" else np.where(labels == int(src))[0]
284         pick = np.random.RandomState(int(seed)).choice(pool, size=min(n, len(pool)), replace=False)
285         batch = [smart_preprocess_to_8x8(np.clip(images[i], 0, 16) / 16 * 255) if pre_b else images[i].astype(float) for i in pick]
286         with st.spinner("Sending batch..."):
287             r = client.predict_batch(batch)
288             if not r.ok:
289                 st.error(r.message)
290             else:
291                 rows = []
292                 for i, res in zip(pick, r.data["results"]):
293                     if "error" in res:
294                         rows.append({"Index": int(i), "True": int(labels[i]), "Predicted": None, "Confidence %": None, "Correct": False})
295                     else:
296                         rows.append({"Index": int(i), "True": int(labels[i]), "Predicted": res["predicted_digit"],
297                                     "Confidence %": round(res["confidence"] * 100, 1), "Correct": res["predicted_digit"] == int(labels[
298 l, i])})
299                 st.session_state.batch = (pd.DataFrame(rows), r.latency_ms, r.status)
300 if st.session_state.get("batch") is not None:
301     df, ms, status = st.session_state.batch
302     m1, m2, m3, m4 = st.columns(4)
303     m1.metric("Accuracy", f"{df['Correct'].mean()*100:.1f}%", f"{int(df['Correct'].sum())} of {len(df)} correct")
304     m2.metric("Mean confidence", f"{df['Confidence %'].mean():.1f}%")
305     m3.metric("Batch round-trip", f"{ms:.0f} ms")
306     m4.metric("Per image", f"{ms/len(df):.1f} ms", f"HTTP {status}")
307     st.dataframe(df.style.apply(lambda row: ["background-color:#fde2e2" if not row["Correct"] else "" for _ in row], axis=1),
308                  width="stretch", height=300, hide_index=True,
309                  column_config={"Confidence %": st.column_config.NumberColumn(format="%.1f"),
310                                "Correct": st.column_config.CheckboxColumn(disabled=True)})
311
312 # ===== API explorer =====
313 ZERO = [0, 0, 5, 13, 9, 1, 0, 0, 0, 13, 15, 10, 15, 5, 0, 0, 3, 15, 2, 0, 11, 8, 0, 0, 4, 12, 10, 0, 8, 8, 0,
314         0, 5, 8, 0, 0, 9, 8, 0, 0, 4, 11, 0, 1, 12, 7, 0, 0, 2, 14, 5, 10, 12, 0, 0, 6, 13, 10, 0, 0, 0]
315 PRESETS = {
316     "GET / (API information)": ("GET", "/", ""),
317     "GET /health": ("GET", "/health", ""),
318     "POST /predict - valid image": ("POST", "/predict", json.dumps({"image": ZERO})),
319     "POST /predict - missing 'image' field": ("POST", "/predict", "{}"),
320     "POST /predict - wrong size (3 values)": ("POST", "/predict", json.dumps({"image": [1, 2, 3]})),
321     "POST /predict - pixel value out of range (99)": ("POST", "/predict", json.dumps({"image": [99] + ZERO[1:]})),
322     "POST /predict - invalid JSON": ("POST", "/predict", '{"image": [1, 2, ]}'),
323     "POST /predict/batch - two valid images": ("POST", "/predict/batch", json.dumps({"images": [ZERO, ZERO]})),
324     "POST /predict/batch - one bad image in the batch": ("POST", "/predict/batch", json.dumps({"images": [ZERO, [1, 2, 3]]})),
325     "POST /predict/batch - empty list": ("POST", "/predict/batch", json.dumps({"images": []})),
326     "GET /nonexistent (expect 404)": ("GET", "/nonexistent", ""),
327     "GET /predict (wrong method, expect 405)": ("GET", "/predict", ""),

```

```

325 }
326 with tab.explorer:
327     st.markdown("Send any request to the Flask API and inspect exactly what comes back – including deliberately broken ones.")
328     choice = st.selectbox("Preset request", list(PRESETS))
329     method, path, body = PRESETS[choice]
330     a, b = st.columns([1, 3])
331     a.text_input("Method", method, disabled=True)
332     b.text_input("Path", path, disabled=True)
333     body_text = st.text_area("Request body", body, height=110, disabled=(method == "GET"), key=f"body_{choice}")
334     if st.button("Send Request", type="primary", disabled=False):
335         r = client.custom(method, path, body_text if method != "GET" else None)
336         st.session_state.explorer = r
337         r = st.session_state.get("explorer")
338         if r is not None:
339             if r.unreachable:
340                 st.error(r.message)
341             else:
342                 good = r.status < 300
343                 st.markdown(f"***HTTP {r.status}** · {r.latency_ms:.1f} ms · "
344                             + ("<U+2705> success" if good else "<U+FE0F> handled error (structured JSON, server stays up)"))
345                 st.json(r.data)
346
347 # ===== Request log =====
348 with tab.log:
349     log = st.session_state.api_log
350     if not log:
351         st.info("No requests yet. Make a prediction and every call to the Flask API appears here.")
352     else:
353         df = pd.DataFrame(log)
354         ok = df["status"].apply(lambda s: isinstance(s, int) and s < 300)
355         m1, m2, m3, m4 = st.columns(4)
356         m1.metric("Requests", len(df)); m2.metric("Successful", int(ok.sum()))
357         m3.metric("Mean latency", f"{df['latency_ms'].mean():.1f} ms"); m4.metric("Slowest", f"{df['latency_ms'].max():.1f} ms")
358         st.dataframe(df.iloc[::-1], width="stretch", hide_index=True, height=480)
359         c1, c2 = st.columns(2)
360         c1.download_button("Download log (CSV)", df.to_csv(index=False), "api_request_log.csv", "text/csv", width="stretch")
361         if c2.button("Clear log", width="stretch"):
362             st.session_state.api_log = []
363             st.rerun()
364
365 st.markdown('<div class="footer">Task 8 · Streamlit frontend integrated with the Flask API from Task 4</div>', unsafe_allow_html=True)

```

5.5 tests/run_integration_tests.py — API integration test suite (262 lines)

```

1 """
2 run_integration_tests.py
3 =====
4 Task 8 test report generator. Starts the REAL Flask API (flask_api/app.py, unchanged from Task 4) in a
5 subprocess and exercises it through the same DigitAPIClient the Streamlit frontend uses.
6
7 Covers: functional behaviour, error handling, faithfulness (API == local model), accuracy through the API,
8 latency, concurrency and crash recovery. Writes tests/results/test_results.json and flask_server.log.
9
10 Run: python tests/run_integration_tests.py
11 """
12 import importlib.util
13 import json
14 import os
15 import signal
16 import statistics
17 import subprocess
18 import sys
19 import time
20 from concurrent.futures import ThreadPoolExecutor
21
22 import numpy as np
23 from sklearn.datasets import load_digits
24
25 HERE = os.path.dirname(os.path.abspath(__file__))
26 ROOT = os.path.dirname(HERE)
27 sys.path.insert(0, os.path.join(ROOT, "frontend"))
28 from api_client import DigitAPIClient # noqa: E402
29 from preprocessing import smart_preprocess_to_8x8, tta_variants # noqa: E402
30
31 URL = "http://127.0.0.1:5000"
32 RESULTS = os.path.join(HERE, "results")
33 os.makedirs(RESULTS, exist_ok=True)
34
35 _server = None
36
37
38 def start_server():
39     global _server
40     log = open(os.path.join(RESULTS, "flask_server.log"), "a")
41     _server = subprocess.Popen([sys.executable, "app.py"], cwd=os.path.join(ROOT, "flask_api"),
42                               stdout=log, stderr=subprocess.STDOUT, preexec_fn=os.setsid)
43     t0 = time.time()
44     while time.time() - t0 < 20:
45         if DigitAPIClient(URL, 1).health().ok:
46             return time.time() - t0
47         time.sleep(0.2)
48     raise RuntimeError("Flask API did not start")
49
50
51 def stop_server():
52     global _server
53     if _server is not None:
54         try:
55             os.killpg(os.getpgid(_server.pid), signal.SIGTERM)
56             _server.wait(timeout=5)
57         except Exception:
58             pass
59     _server = None
60
61
62 # -----

```

```

63 digits = load_digits()
64 IMAGES, LABELS = digits.images, digits.target
65 perm = np.random.default_rng(42).permutation(len(IMAGES))
66 HELD_OUT = perm[int(len(IMAGES) * 0.85):] # 270 images, same split as Task 7
67
68 spec = importlib.util.spec_from_file_location("flask_app", os.path.join(ROOT, "flask_api", "app.py"))
69 flask_app = importlib.util.module_from_spec(spec)
70 spec.loader.exec_module(flask_app) # gives us the API's own model + parse_image for comparison
71 local_model = flask_app.model
72
73 tests, metrics = [], {}
74 client = DigitAPIClient(URL, 5)
75
76
77 def run(tid, group, name, expected, fn):
78     t0 = time.perf_counter()
79     try:
80         ok, actual = fn()
81     except Exception as e: # a crashing test is a failed test
82         ok, actual = False, f"exception: {e!r}"
83     ms = (time.perf_counter() - t0) * 1000
84     tests.append({"id": tid, "group": group, "name": name, "expected": expected, "actual": actual,
85                 "passed": bool(ok), "ms": round(ms, 1)})
86     print(f"[{'PASS' if ok else 'FAIL'}] {tid} {name} -> {actual}")
87
88
89 def first_of(d):
90     return int(np.where(LABELS == d)[0][0])
91
92
93 def api_predict_many(indices, preprocess=False):
94     preds, probs = [], []
95     for s in range(0, len(indices), 100):
96         chunk = indices[s:s + 100]
97         imgs = [smart_preprocess_to_8x8(np.clip(IMAGES[i], 0, 16) / 16 * 255) if preprocess else IMAGES[i].astype(float) for i in ch
98         unk]
99         r = client.predict_batch(imgs)
100         assert r.ok, r.message
101         for res in r.data["results"]:
102             preds.append(res["predicted_digit"]); probs.append([res["probabilities"][str(d)] for d in range(10)])
103     return np.array(preds), np.array(probs)
104
105
106 def main():
107     boot = start_server()
108     metrics["server_start_s"] = round(boot, 2)
109     print(f"Flask API started in {boot:.2f} s\n")
110     zero = [0, 0, 5, 13, 9, 1, 0, 0, 0, 0, 13, 15, 10, 15, 5, 0, 0, 3, 15, 2, 0, 11, 8, 0, 0, 4, 12, 0, 0, 8, 8, 0,
111            0, 5, 8, 0, 0, 9, 8, 0, 0, 4, 11, 0, 1, 12, 7, 0, 0, 2, 14, 5, 10, 12, 0, 0, 0, 0, 6, 13, 10, 0, 0, 0]
112
113     # ----- A. connection and information
114     run("T01", "Connection", "GET / returns the API description", "200 + 3 endpoints listed",
115         lambda: ((r := client.info()).ok and len(r.data["endpoints"]) == 3, f"HTTP {r.status}, endpoints={len(r.data['endpoints'])}"))
116
117     run("T02", "Connection", "GET /health reports the model as loaded", "200, model loaded = true",
118         lambda: ((r := client.health()).ok and r.data["model_loaded"] is True, f"HTTP {r.status}, {r.data}"))
119
120     # ----- B. predictions
121     def t03():
122         i = first_of(3); r = client.predict(IMAGES[i])
123         s = sum(r.data["probabilities"].values())
124         return r.ok and r.data["predicted_digit"] == 3 and abs(s - 1) < 1e-3, f"HTTP {r.status}, digit {r.data['predicted_digit']} @ {r.data['confidence']}"
125     run("T03", "Prediction", "Single raw image (true 3) via POST /predict", "digit 3, probabilities sum to 1", t03)
126
127     def t04():
128         i = first_of(7); px = smart_preprocess_to_8x8(np.clip(IMAGES[i], 0, 16) / 16 * 255); r = client.predict(px)
129         return r.ok and r.data["predicted_digit"] == 7, f"digit {r.data['predicted_digit']} @ {r.data['confidence']}"
130     run("T04", "Prediction", "Frontend-preprocessed image (true 7) via POST /predict", "digit 7", t04)
131
132     def t05():
133         r16 = client.custom("POST", "/predict", json.dumps({"image": zero}))
134         r01 = client.custom("POST", "/predict", json.dumps({"image": [v / 16 for v in zero]}))
135         same = r16.data["predicted_digit"] == r01.data["predicted_digit"] == 0
136         return same and r16.ok and r01.ok, f"0-16 scale -> {r16.data['predicted_digit']}, 0-1 scale -> {r01.data['predicted_digit']}"
137     run("T05", "Prediction", "0-16 and 0-1 pixel scales give the same digit", "both predict 0", t05)
138
139     run("T06", "Prediction", "Nested 8x8 list is accepted", "200, digit 0",
140         lambda: ((r := client.custom("POST", "/predict", json.dumps({"image": [zero[k:k + 8] for k in range(0, 64, 8)]}))).ok and r.data["predicted_digit"] == 0,
141             f"HTTP {r.status}, digit {r.data.get('predicted_digit')}"))
142
143     def t07():
144         idx = [first_of(3), first_of(7), first_of(8)]; r = client.predict_batch([IMAGES[i] for i in idx])
145         got = [x["predicted_digit"] for x in r.data["results"]]
146         return r.ok and got == [3, 7, 8], f"HTTP {r.status}, predicted {got}, count={r.data['count']}"
147     run("T07", "Prediction", "Batch of 3 real digits (3, 7, 8)", "[3, 7, 8]", t07)
148
149     def t08():
150         variants = tta_variants(IMAGES[first_of(8)].astype(float)); r = client.predict_batch(variants)
151         probs = np.mean([x["probabilities"][str(d)] for d in range(10)] for x in r.data["results"], axis=0)
152         return r.ok and r.data["count"] == 11 and abs(probs.sum() - 1) < 1e-3 and int(probs.argmax()) == 8, \
153             f"{r.data['count']} variants, averaged digit {int(probs.argmax())} @ {probs.max():.4f}"
154     run("T08", "Prediction", "Test-time augmentation: 11 variants in one batch call", "11 results, averaged digit 8", t08)
155
156     def t09():
157         r = client.predict_batch([IMAGES[i] for i in range(100)])
158         return r.ok and r.data["count"] == 100, f"HTTP {r.status}, count={r.data['count']}"
159     run("T09", "Prediction", "Batch at the limit (100 images)", "200, 100 results", t09)
160
161     # ----- C. error handling
162     def err(method, path, body, status, contains):
163         r = client.custom(method, path, body)
164         msg = (r.data or {}).get("error", "")
165         return r.status == status and contains in msg, f"HTTP {r.status}: {msg}"
166     run("T10", "Errors", "Missing 'image' field", "400", lambda: err("POST", "/predict", "{}", 400, "Missing 'image'"))
167     run("T11", "Errors", "Wrong-size image (3 values)", "400", lambda: err("POST", "/predict", '{"image": [1,2,3]}', 400, "got 3 valu
168     es"))
169     run("T12", "Errors", "Invalid JSON body", "400", lambda: err("POST", "/predict", '{"image": [1, 2, ', 400, "valid JSON"))
170     run("T13", "Errors", "Pixel value above range (99)", "400", lambda: err("POST", "/predict", json.dumps({"image": [99] + zero[1:]

```

```

    }, 400, "within [0, 16]"))
168 run("T14", "Errors", "Negative pixel value", "400", lambda: err("POST", "/predict", json.dumps({"image": [-1] + zero[1:]}), 400,
    "within"))
169 run("T15", "Errors", "Non-numeric pixel values", "400", lambda: err("POST", "/predict", json.dumps({"image": ["a"] * 64}), 400,
    ""))
170 run("T16", "Errors", "Unknown endpoint", "404", lambda: err("GET", "/nonexistent", None, 404, "not found"))
171 run("T17", "Errors", "Wrong HTTP method (GET /predict)", "405", lambda: err("GET", "/predict", None, 405, "Method not allowed"))
172 run("T18", "Errors", "Empty batch list", "400", lambda: err("POST", "/predict/batch", '{"images": []}', 400, "non-empty"))
173 run("T19", "Errors", "Batch larger than 100", "400", lambda: err("POST", "/predict/batch", json.dumps({"images": [zero] * 101}),
    "limited to 100"))
174
175 def t20():
176     r = client.custom("POST", "/predict/batch", json.dumps({"images": [zero, [1, 2, 3], zero]}))
177     res = r.data["results"]
178     return r.status == 200 and "error" in res[1] and "predicted_digit" in res[0] and "predicted_digit" in res[2], \
179         f"HTTP {r.status}: entries -> {[ 'ok' if 'predicted_digit' in x else 'error' for x in res]}"
180 run("T20", "Errors", "One bad image does not fail the whole batch", "200; entries [ok, error, ok]", t20)
181
182 # ----- D. faithfulness and accuracy through the API
183 def t21():
184     allidx = np.arange(len(IMAGES)); preds, probs = api_predict_many(allidx)
185     x = np.stack([flask_app.parse_image(IMAGES[i].flatten().tolist())[0] for i in allidx])
186     lp = local_model.predict_proba(x)
187     same = int((lp.argmax(1) == preds).sum()); maxdiff = float(np.abs(lp - probs).max())
188     metrics["consistency"] = {"images": len(allidx), "same_prediction": same, "max_abs_prob_diff": maxdiff}
189     return same == len(allidx) and maxdiff < 1e-4, f"{same}/{len(allidx)} identical predictions, max |p_api - p_local| = {maxdiff}
190 f:.1e}"
191 run("T21", "Faithfulness", "API output equals the local model on all 1,797 images", "identical predictions, diff < 1e-4", t21)
192
193 def t22():
194     out = {}
195     for name, idx in (("held_out_270", HELD_OUT), ("all_1797", np.arange(len(IMAGES)))):
196         for pre in (False, True):
197             p, _ = api_predict_many(idx, pre)
198             out[f"{name}_{'preprocessed' if pre else 'raw'}"] = round(float((p == LABELS[idx]).mean()) * 100, 2)
199     metrics["accuracy_via_api"] = out
200     return out["held_out_270_preprocessed"] > 95 and out["held_out_270_raw"] > 90, json.dumps(out)
201 run("T22", "Accuracy", "Accuracy measured through the HTTP API", "> 90 % raw, > 95 % preprocessed (held-out)", t22)
202
203 # ----- E. latency, throughput, concurrency
204 def t23():
205     sel = np.random.default_rng(0).choice(HELD_OUT, 200)
206     lat = []
207     for i in sel:
208         r = client.predict(IMAGES[i]); assert r.ok; lat.append(r.latency_ms)
209     a = np.array(lat)
210     m = {"n": 200, "mean_ms": round(a.mean(), 1), "median_ms": round(float(np.median(a)), 1), "p95_ms": round(float(np.percentil
211 e(a, 95)), 1),
212         "p99_ms": round(float(np.percentile(a, 99)), 1), "max_ms": round(a.max(), 1), "min_ms": round(a.min(), 1)}
213     metrics["latency_single"] = m
214     return m["p95_ms"] < 250, f"median {m['median_ms']} ms, p95 {m['p95_ms']} ms, max {m['max_ms']} ms"
215 run("T23", "Performance", "200 sequential POST /predict calls", "p95 round-trip < 250 ms", t23)
216
217 def t24():
218     times = []
219     for _ in range(5):
220         r = client.predict_batch([IMAGES[i] for i in range(100)]); assert r.ok; times.append(r.latency_ms)
221     m = {"batch_size": 100, "median_ms": round(float(np.median(times)), 1), "per_image_ms": round(float(np.median(times)) / 100,
222 2)}
223     metrics["latency_batch"] = m
224     return True, f"100 images in {m['median_ms']} ms ({m['per_image_ms']} ms per image)"
225 run("T24", "Performance", "POST /predict/batch with 100 images (median of 5)", "completes; per-image cost reported", t24)
226
227 def t25():
228     sel = np.random.default_rng(1).choice(HELD_OUT, 200)
229     def one(i):
230         c = DigitAPIClient(URL, 10); r = c.predict(IMAGES[i]); return r.ok and r.data["predicted_digit"] is not None, r.latency_
231 ms
232     t0 = time.perf_counter()
233     with ThreadPoolExecutor(8) as ex:
234         res = list(ex.map(one, sel))
235     wall = time.perf_counter() - t0
236     okn = sum(1 for o, _ in res if o)
237     metrics["concurrency"] = {"workers": 8, "requests": 200, "succeeded": okn, "wall_s": round(wall, 2), "throughput_rps": round
238 (200 / wall, 1)}
239     return okn == 200, f"{okn}/200 succeeded with 8 parallel clients, {200/wall:.1f} requests/s"
240 run("T25", "Performance", "200 requests from 8 parallel clients", "all 200 succeed", t25)
241
242 # ----- F. resilience
243 def t26():
244     stop_server(); time.sleep(0.5)
245     h, p = client.health(), client.predict(IMAGES[0])
246     return h.unreachable and p.unreachable and h.error and p.error, f"health -> '{h.message}' (no exception raised)"
247 run("T26", "Resilience", "API stopped: client reports an error instead of crashing", "unreachable result with message", t26)
248
249 def t27():
250     t0 = time.time(); start_server(); rec = time.time() - t0
251     r = client.predict(IMAGES[first_of(3)]); metrics["recovery_s"] = round(rec, 2)
252     return r.ok and r.data["predicted_digit"] == 3, f"API restarted; first prediction OK after {rec:.2f} s"
253 run("T27", "Resilience", "API restarted: the same client works again", "prediction succeeds", t27)
254
255 passed = sum(t["passed"] for t in tests)
256 metrics["passed"], metrics["total"] = passed, len(tests)
257 json.dump({"tests": tests, "metrics": metrics}, open(os.path.join(RESULTS, "test_results.json"), "w"), indent=2)
258 print(f"\n{passed}/{len(tests)} tests passed")
259 print(json.dumps(metrics, indent=2))
260 stop_server()
261
262 if __name__ == "__main__":
263     try:
264         main()
265     finally:
266         stop_server()

```

5.6 tests/e2e_ui_tests.py — Browser end-to-end test and screenshot capture (255 lines)

```
1 """
2 e2e_ui_tests.py
3 =====
4 End-to-end test of the integrated application: a real browser (Playwright / Chromium) drives the Streamlit
5 frontend, which calls the real Flask API. Each step checks what the user sees against what the API returns,
6 and saves a screenshot. Writes tests/results/ui_test_results.json, screens/*.png and e2e_flask_server.log.
7
8 Requires: pip install playwright && playwright install chromium
9 Run:      python tests/e2e_ui_tests.py
10 """
11 import asyncio
12 import json
13 import math
14 import os
15 import random
16 import re
17 import signal
18 import subprocess
19 import sys
20 import time
21
22 import numpy as np
23 from playwright.async_api import async_playwright
24
25 HERE = os.path.dirname(os.path.abspath(__file__))
26 ROOT = os.path.dirname(HERE)
27 RES = os.path.join(HERE, "results"); SCR = os.path.join(RES, "screens")
28 os.makedirs(SCR, exist_ok=True)
29 sys.path.insert(0, os.path.join(ROOT, "frontend"))
30 from api_client import DigitAPIClient          # noqa: E402
31 from preprocessing import smart_preprocess_to_8x8 # noqa: E402
32 from sklearn.datasets import load_digits        # noqa: E402
33
34 CHROME = os.environ.get("CHROME_PATH", "/opt/pw-browsers/chromium-1194/chrome-linux/chrome")
35 API, UI_PORT = "http://127.0.0.1:5000", 8610
36 UI = f"http://localhost:{UI_PORT}"
37 PHOTO = os.path.join(HERE, "sample_photo.png")
38 procs = {}
39 results = []
40
41
42 def start_flask():
43     log = open(os.path.join(RES, "e2e_flask_server.log"), "a")
44     procs["flask"] = subprocess.Popen([sys.executable, "app.py"], cwd=os.path.join(ROOT, "flask_api"), stdout=log,
45                                       stderr=subprocess.STDOUT, preexec_fn=os.setsid)
46     for _ in range(100):
47         if DigitAPIClient(API, 1).health().ok:
48             return
49         time.sleep(0.2)
50     raise RuntimeError("Flask did not start")
51
52
53 def stop(name):
54     p = procs.pop(name, None)
55     if p:
56         try:
57             os.killpg(os.getpgid(p.pid), signal.SIGTERM); p.wait(timeout=5)
58         except Exception:
59             pass
60
61
62 def start_streamlit():
63     env = dict(os.environ, DIGIT_API_URL=API)
64     procs["st"] = subprocess.Popen([sys.executable, "-m", "streamlit", "run", "frontend/streamlit_app.py", "--server.headless", "tru
65 e",
66                                     "--server.port", str(UI_PORT), "--browser.gatherUsageStats", "false"], cwd=ROOT, env=env,
67                                     stdout=open(os.path.join(RES, "streamlit.log"), "w"), stderr=subprocess.STDOUT, preexec_fn=os.set
68 sid)
69     time.sleep(6)
70
71
72 def record(tid, name, expected, ok, actual):
73     results.append({"id": tid, "name": name, "expected": expected, "actual": actual, "passed": bool(ok)})
74     print(f"[{'PASS' if ok else 'FAIL'}] {tid} {name} -> {actual}")
75
76
77 async def settle(pg, ms=1800):
78     await pg.wait_for_timeout(ms)
79
80
81 async def to_top(pg):
82     await pg.mouse.move(2, 2)
83     await pg.evaluate("""() => { for (const s of ['[data-testid="stMain"]','section.main','stApp']) { const e=document.querySelector
84 r(s); if(e) e.scrollTop=0; } window.scrollTo(0,0); }""")
85     await pg.wait_for_timeout(250)
86
87
88 async def fit(pg, width=1280):
89     await pg.set_viewport_size({"width": width, "height": 1000}); await pg.wait_for_timeout(700)
90     await to_top(pg)
91     h = await pg.evaluate("""() => { const m=document.querySelector('[data-testid="stMainBlockContainer"]'); return Math.ceil(m.getB
92 oundingClientRect().bottom + 90); }""")
93     await pg.set_viewport_size({"width": width, "height": max(h, 900)})
94     await pg.wait_for_timeout(900)
95
96
97 async def shot_main(pg, name):
98     await fit(pg)
99     await pg.locator('[data-testid="stMainBlockContainer"]').screenshot(path=os.path.join(SCR, name))
100
101
102 async def shot_full(pg, name, width=1280):
103     await fit(pg, width)
104     await pg.screenshot(path=os.path.join(SCR, name))
105
106
107 def status_line(side):
108     m = re.findall(r"API (?:(online|offline)[^\n]*|model loaded[^\n]*|Cannot reach[^\n]*", side)
```

```

105     return " · ".join(x.strip() for x in m)
106
107
108 async def visible_metrics(pg):
109     return await pg.evaluate("""() => [...document.querySelectorAll('[data-testid="stMetric"]')]
    .filter(e => e.offsetParent !== null
    & ).map(e => e.innerText.replace(/\\n/g, ' '))""")
110
111
112 async def pick_select(pg, container_idx, option):
113     box = pg.locator('[data-testid="stSelectbox"]:visible').nth(container_idx)
114     await box.click()
115     await pg.get_by_role("option", name=option, exact=True).click()
116     await settle(pg, 1200)
117
118
119 async def stroke(pg, box, pts):
120     px = lambda p: (box["x"] + p[0] * box["width"], box["y"] + p[1] * box["height"])
121     await pg.mouse.move(px(pts[0])); await pg.mouse.down()
122     for p in pts[1:]:
123         await pg.mouse.move(px(p), steps=2)
124     await pg.mouse.up()
125
126
127 def arc(cx, cy, rx, ry, a0, a1, n=50, seed=0):
128     r = random.Random(seed)
129     return [(cx + rx * math.cos(math.radians(a0 + (a1 - a0) * i / n)) + r.uniform(-.004, .004),
130             cy + ry * math.sin(math.radians(a0 + (a1 - a0) * i / n)) + r.uniform(-.004, .004)) for i in range(n + 1)]
131
132
133 async def run():
134     digits = load_digits(); IM, LB = digits.images, digits.target
135     direct = DigitAPIClient(API, 5)
136     start_flask(); start_streamlit()
137     async with async_playwright() as p:
138         b = await p.chromium.launch(executable_path=CHROME, args=["--no-sandbox"])
139         pg = await b.new_page(viewport={"width": 1280, "height": 1000}, device_scale_factor=2)
140         await pg.goto(UI, wait_until="networkidle"); await settle(pg, 3000)
141         try:
142             await steps(pg, digits, IM, LB, direct)
143         except Exception:
144             await pg.screenshot(path=os.path.join(SCR, "debug_failure.png"))
145             raise
146         await b.close()
147     json.dump(results, open(os.path.join(RES, "ui_test_results.json"), "w"), indent=2)
148     print(f"\n{sum(r['passed'] for r in results)}/{len(results)} UI checks passed")
149
150
151 async def steps(pg, digits, IM, LB, direct):
152     if True:
153         # U01 connection status
154         side = await pg.locator('[data-testid="stSidebar"]').inner_text()
155         record("U01", "App loads and the sidebar shows the Flask API as online", "'API online' in sidebar", "API online" in side, st
156         & atus_line(side))
157         await shot_full(pg, "u01_home_online.png")
158
159         # U02 sample prediction (digit 7) via the UI == direct API call
160         await pick_select(pg, 0, "7")
161         idx = int(np.where(LB == 7)[0][0])
162         await pg.get_by_role("button", name="<U+1F680> Send to Flask API").click()
163         await pg.get_by_text("Result from the Flask API").wait_for(timeout=15000); await settle(pg)
164         ui_digit = int(await pg.locator(".result-digit").first.inner_text())
165         api_digit = direct.predict(IM[idx].astype(float)).data["predicted_digit"]
166         await shot_full(pg, "u02_full.png")
167         await pg.get_by_text("Show the raw request and response").click(); await settle(pg, 800)
168         record("U02", "Sample digit 7: digit shown in the UI equals the API's answer", f"UI = API = 7", ui_digit == api_digit == 7,
169         & f"UI shows {ui_digit}, API returns {api_digit}")
170         await shot_main(pg, "u02_sample_result.png")
171
172         # U03 test-time augmentation uses one batch call
173         await pg.get_by_text("Test-time augmentation").first.click(); await settle(pg, 1000)
174         await pg.get_by_role("button", name="<U+1F680> Send to Flask API").click(); await settle(pg, 2500)
175         mets = " · ".join(await visible_metrics(pg))
176         record("U03", "Test-time augmentation sends 11 variants in one /predict/batch request", "endpoint '/predict/batch'", "/predi
177         & ct/batch" in mets, mets[:110])
178         await shot_main(pg, "u03_tta_result.png")
179         await pg.get_by_text("Test-time augmentation").first.click(); await settle(pg, 800)
180
181         # U04 drawing
182         await pg.get_by_text("Draw", exact=True).first.click(); await settle(pg, 3500)
183         cv = pg.locator("canvas.upper-canvas").first
184         box = await cv.bounding_box()
185         await stroke(pg, box, arc(0.47, 0.30, 0.20, 0.17, -150, 90, seed=1)); await stroke(pg, box, arc(0.47, 0.64, 0.23, 0.19, -90,
186         & 150, seed=2))
187         await settle(pg, 3000)
188         await pg.get_by_role("button", name="<U+1F680> Send to Flask API").click(); await pg.wait_for_timeout(3500)
189         ui_digit = int(await pg.locator(".result-digit").first.inner_text())
190         record("U04", "Hand-drawn '3' is sent to the API and the prediction is shown", "digit 3", ui_digit == 3, f"UI shows {ui_digi
191         & t}")
192         await shot_main(pg, "u04_draw_result.png")
193
194         # U05 upload
195         await pg.get_by_text("Upload photo", exact=True).first.click(); await settle(pg, 1500)
196         async with pg.expect_file_choser(timeout=15000) as fc:
197             await pg.get_by_test_id("stFileUploaderDropzone").locator("button").first.click()
198             await (await fc.value).set_files(PHOTO); await settle(pg, 3000)
199             await pg.get_by_role("button", name="<U+1F680> Send to Flask API").click(); await pg.wait_for_timeout(3500)
200             ui_digit = int(await pg.locator(".result-digit").first.inner_text())
201             record("U05", "Uploaded photo of a '2' is preprocessed, sent and classified", "digit 2", ui_digit == 2, f"UI shows {ui_digit
202             & }")
203             await shot_main(pg, "u05_upload_result.png")
204
205         # U06 batch (raw, then preprocessed)
206         await pg.get_by_role("tab", name=re.compile("Batch test")).click(); await settle(pg, 1200)
207         await pg.get_by_role("button", name="▶ Run batch").click(); await settle(pg, 3000)
208         m_raw = await visible_metrics(pg)
209         acc_raw = float(re.search(r"Accuracy \[| \[\\d.]+\]", m_raw[0]).group(1))
210         await shot_main(pg, "u06_batch_raw.png")
211         await pg.get_by_text("Crop → scale → center first").filter(visible=True).first.click(); await settle(pg, 800)
212         await pg.get_by_role("button", name="▶ Run batch").click(); await settle(pg, 3000)
213         m_pre = await visible_metrics(pg)

```

```

209     acc_pre = float(re.search(r"Accuracy \\\ (\\d\\.\\+)%", m_pre[0]).group(1))
210     sel = np.random.RandomState(1).choice(np.arange(len(IM)), size=50, replace=False)
211     direct_res = direct.predict_batch([IM[i].astype(float) for i in sel]).data["results"]
212     exp_raw = round(float(np.mean([r["predicted_digit"] == LB[i] for r, i in zip(direct_res, sel)])) * 100, 1)
213     record("U06", "Batch tab: 50 images in one request; UI accuracy equals a direct API call", f"raw accuracy = {exp_raw}%", abs
↳ (acc_raw - exp_raw) < 0.05,
214         f"UI raw {acc_raw}% | UI preprocessed {acc_pre}% | direct raw {exp_raw}%")
215     await shot_main(pg, "u06_batch_pre.png")
216
217     # U07 API explorer: handled errors
218     await pg.get_by_role("tab", name=re.compile("API explorer")).click(); await settle(pg, 1200)
219     cases = [{"POST /predict - wrong size (3 values)", "u07a_explorer_400.png", 400},
220             {"POST /predict/batch - one bad image in the batch", "u07b_explorer_batch.png", 200},
221             {"GET /nonexistent (expect 404)", "u07c_explorer_404.png", 404}]
222     for preset, fname, code in cases:
223         await pick_select(pg, 0, preset)
224         await pg.get_by_role("button", name="Send request").click(); await settle(pg, 1800)
225         txt = await pg.locator('[data-testid="stMainBlockContainer"]').inner_text()
226         record("U07", f"API explorer: {preset.split(' ')[0]} {preset.split(' ')[-1][:34]} shows HTTP {code}", f"HTTP {code}"
↳ , f"HTTP {code}" in txt, f"page shows 'HTTP {code}'" if f"HTTP {code}" in txt else "status not found")
227         await shot_main(pg, fname)
228
229     # U08 request log
230     await pg.get_by_role("tab", name=re.compile("Request log")).click(); await settle(pg, 1500)
231     mets = await visible_metrics(pg)
232     n_req = int(re.search(r"Requests \\\ (\\d\\.\\+)", mets[0]).group(1))
233     record("U08", "Request log records every call made by the UI", "log has >= 8 requests", n_req >= 8, f"{n_req} requests logge
↳ d; {' / '.join(mets)}":150])
234     await shot_main(pg, "u08_request_log.png")
235
236     # U09 API goes offline, then recovers
237     stop("flask"); await settle(pg, 500)
238     await pg.goto(UI, wait_until="networkidle"); await settle(pg, 3500) # a fresh visit while the API is down
239     side = await pg.locator('[data-testid="stSidebar"]').inner_text()
240     disabled = await pg.get_by_role("button", name="Send to Flask API").is_disabled()
241     record("U09", "API stopped: UI shows 'API offline', explains how to start it, disables Send", "offline banner + disabled but
↳ ton", "API offline" in side and disabled, f"sidebar: {status_line(side)} | Send button disabled = {disabled}")
242     await shot_full(pg, "u09_offline.png", width=1000)
243     start_flask()
244     await pg.get_by_role("button", name="Re-check connection").click(); await settle(pg, 3000)
245     side = await pg.locator('[data-testid="stSidebar"]').inner_text()
246     await pg.get_by_role("button", name="Send to Flask API").click(); await pg.wait_for_timeout(3000)
247     ok = "API online" in side and await pg.locator(".result-digit").count() > 0
248     record("U10", "API restarted: UI reconnects and predictions work again", "'API online' + a result", ok, f"sidebar: {status_l
↳ ine(side)}; a new result is displayed")
249
250
251 if __name__ == "__main__":
252     try:
253         asyncio.run(run())
254     finally:
255         stop("st"); stop("flask")

```

5.7 requirements.txt

```

streamlit>=1.53
flask
requests
numpy
pandas
altair
pillow
scikit-learn
streamlit-drawable-canvas

```

6. Screenshots of the Interface

All screenshots were captured from the running integrated application: the Streamlit frontend on port 8501 calling the Flask API on port 5000, driven by a headless Chromium browser (Playwright) with real clicks, real mouse drawing and a real file chooser. Predictions, probabilities and timings are genuine replies from the Flask server. The uploaded “photo” in Section 6.5 is a synthetic stand-in made from held-out dataset digit #566 (a '2'; upscaled, blurred, rotated and noised). Each screenshot has its own page.

6.1 Connected Frontend and Real-Time Prediction

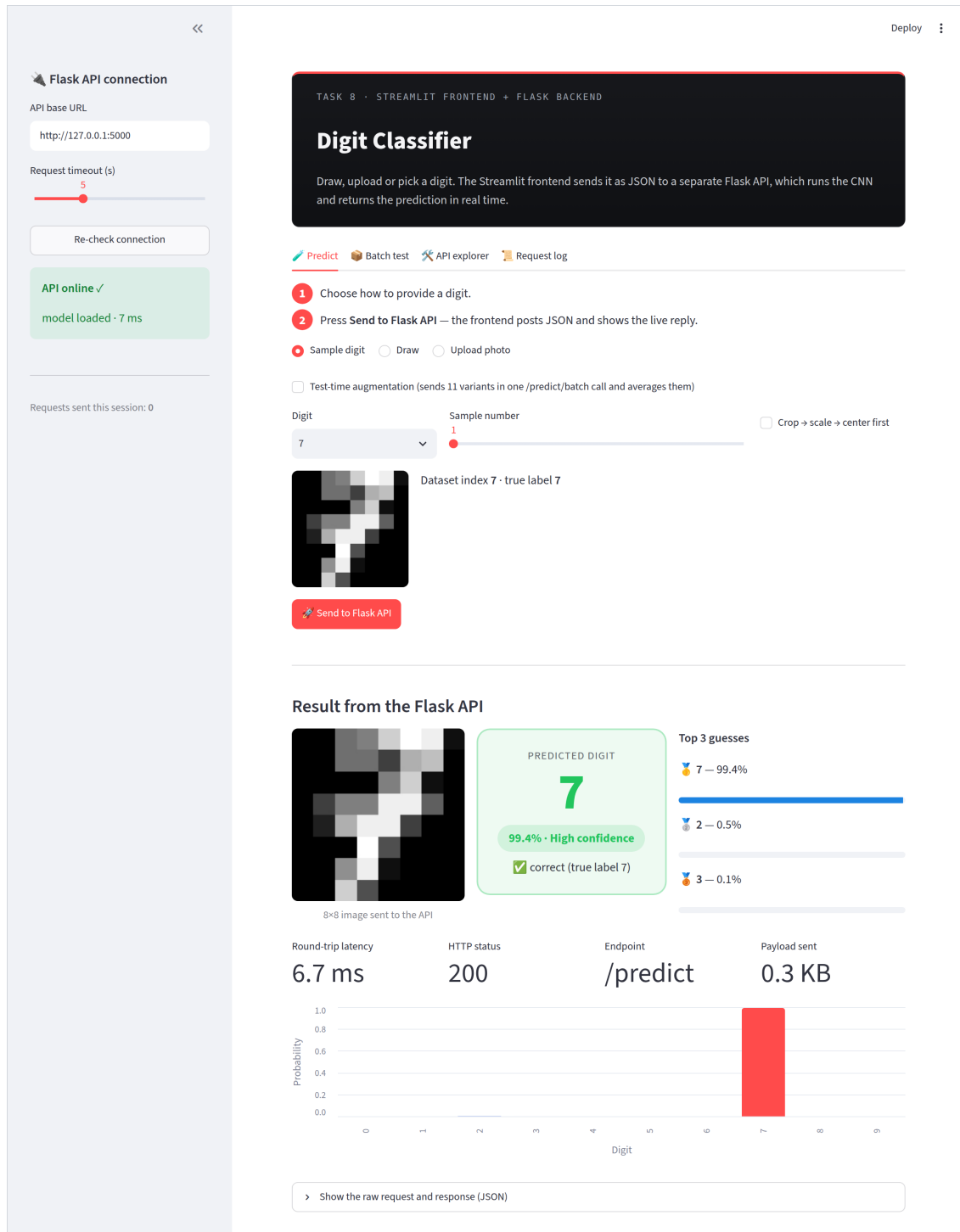
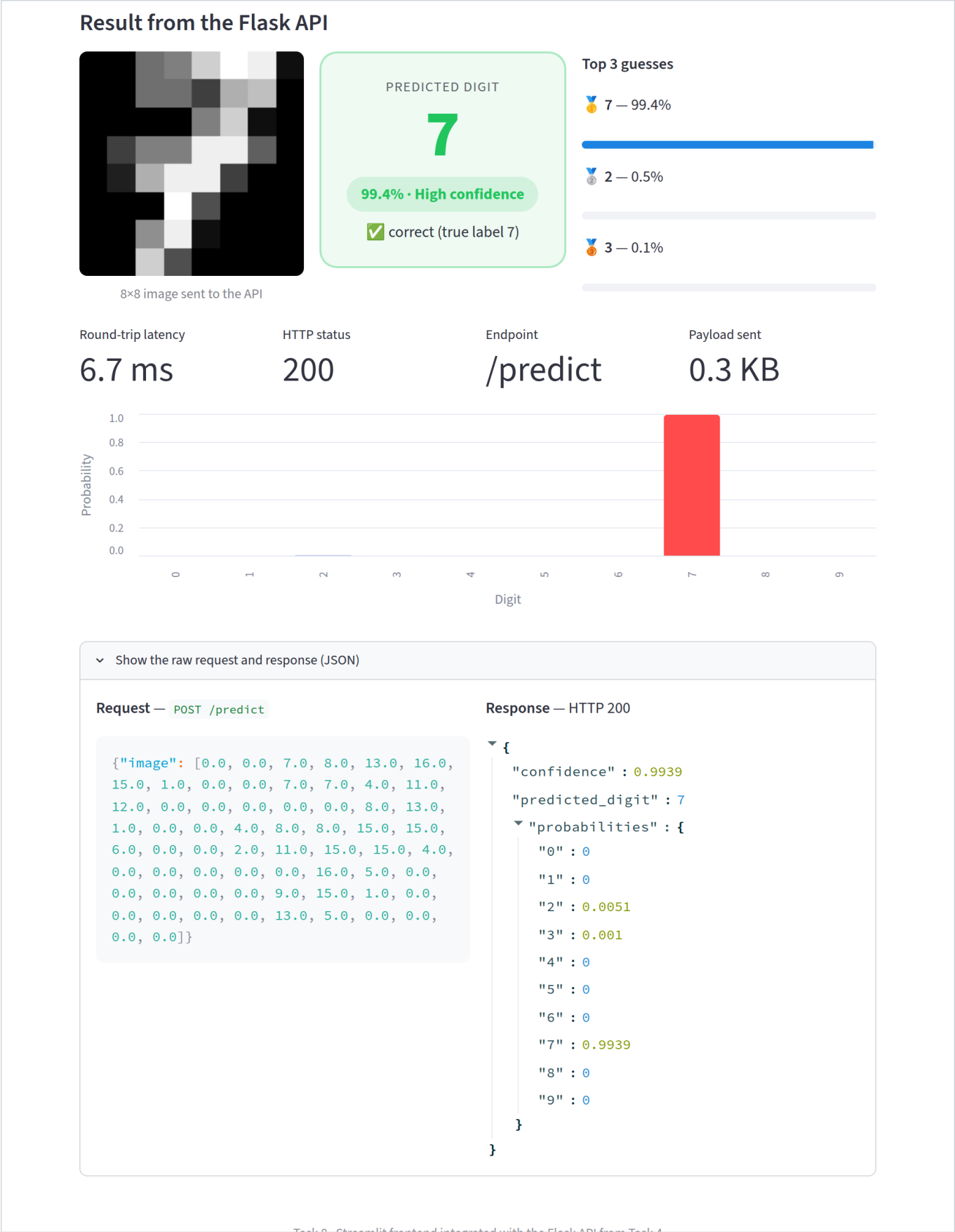
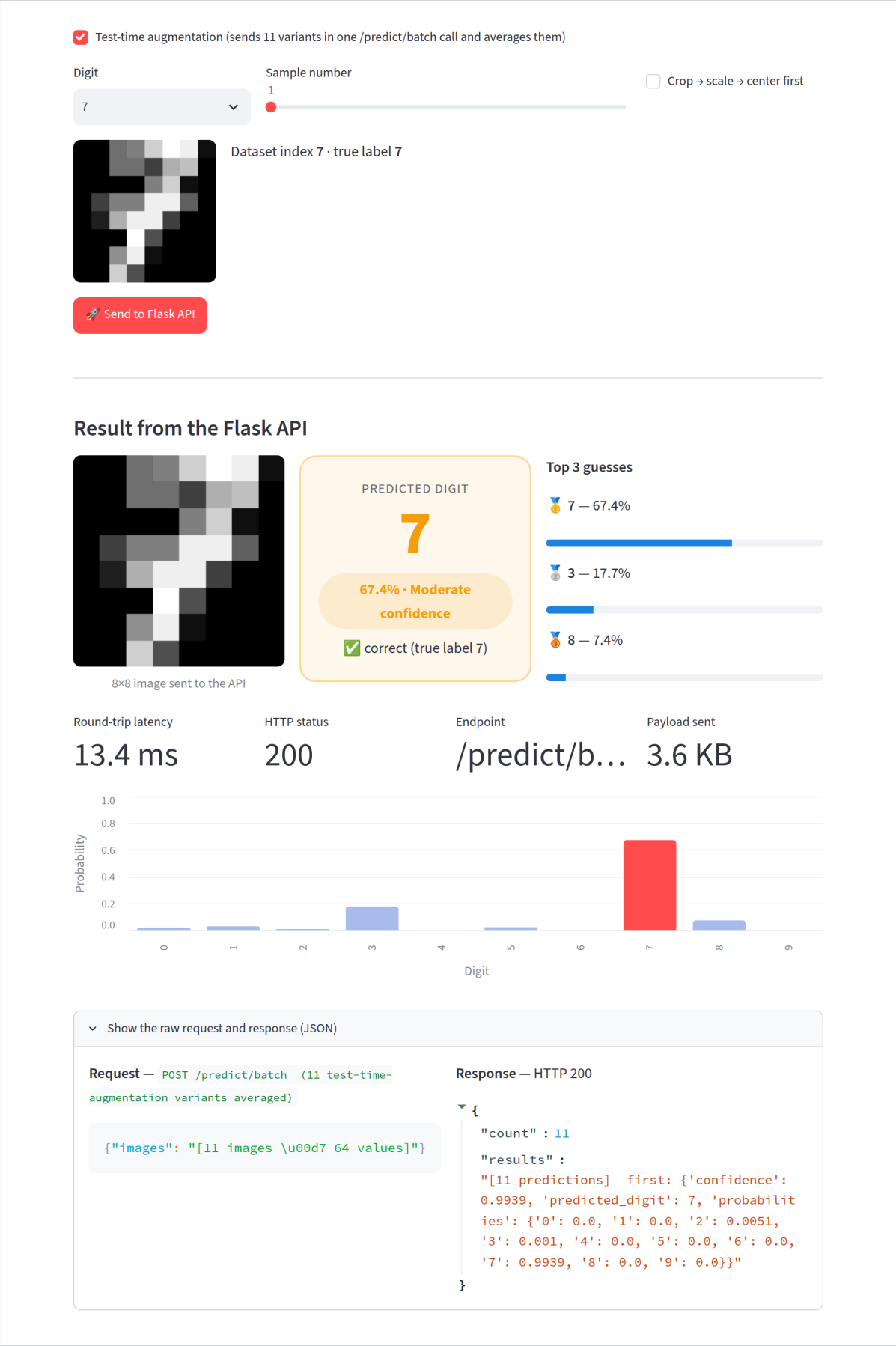


Figure 1: Sidebar: the API is online with the model loaded, and the request counter. Main page: dataset digit 7 sent with one click; the Flask API answers 7 (99.4 %, high confidence), shown with the top-3 guesses, round-trip time, HTTP status, endpoint and payload size.

6.2 Request and Response Shown in the UI



6.3 Test-Time Augmentation through the Batch Endpoint



6.4 Drawing a Digit

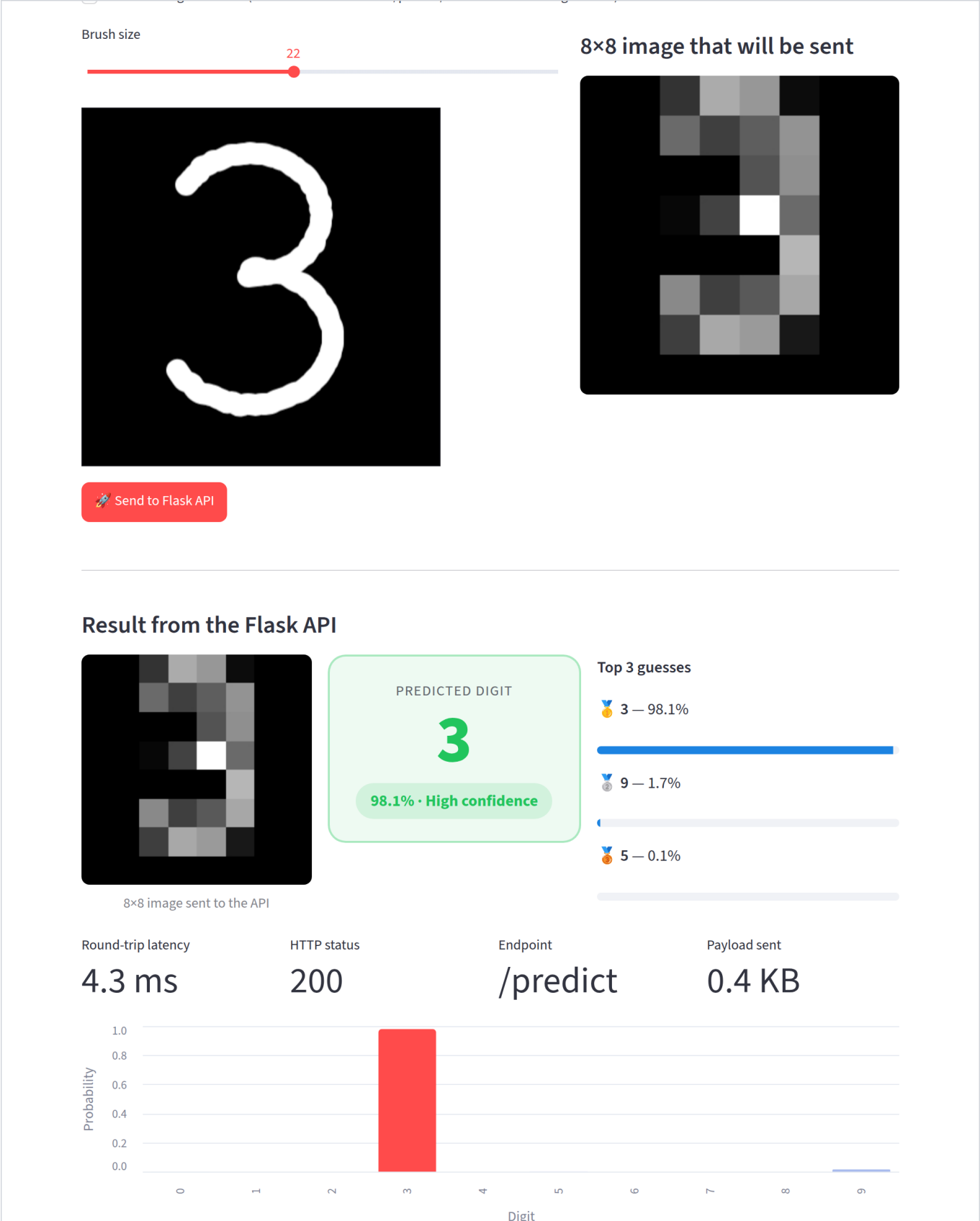


Figure 4: A '3' drawn with the mouse, the live 8×8 image produced by the frontend preprocessing, and the Flask API's reply: predicted 3 with high confidence.

6.5 Uploading a Photo

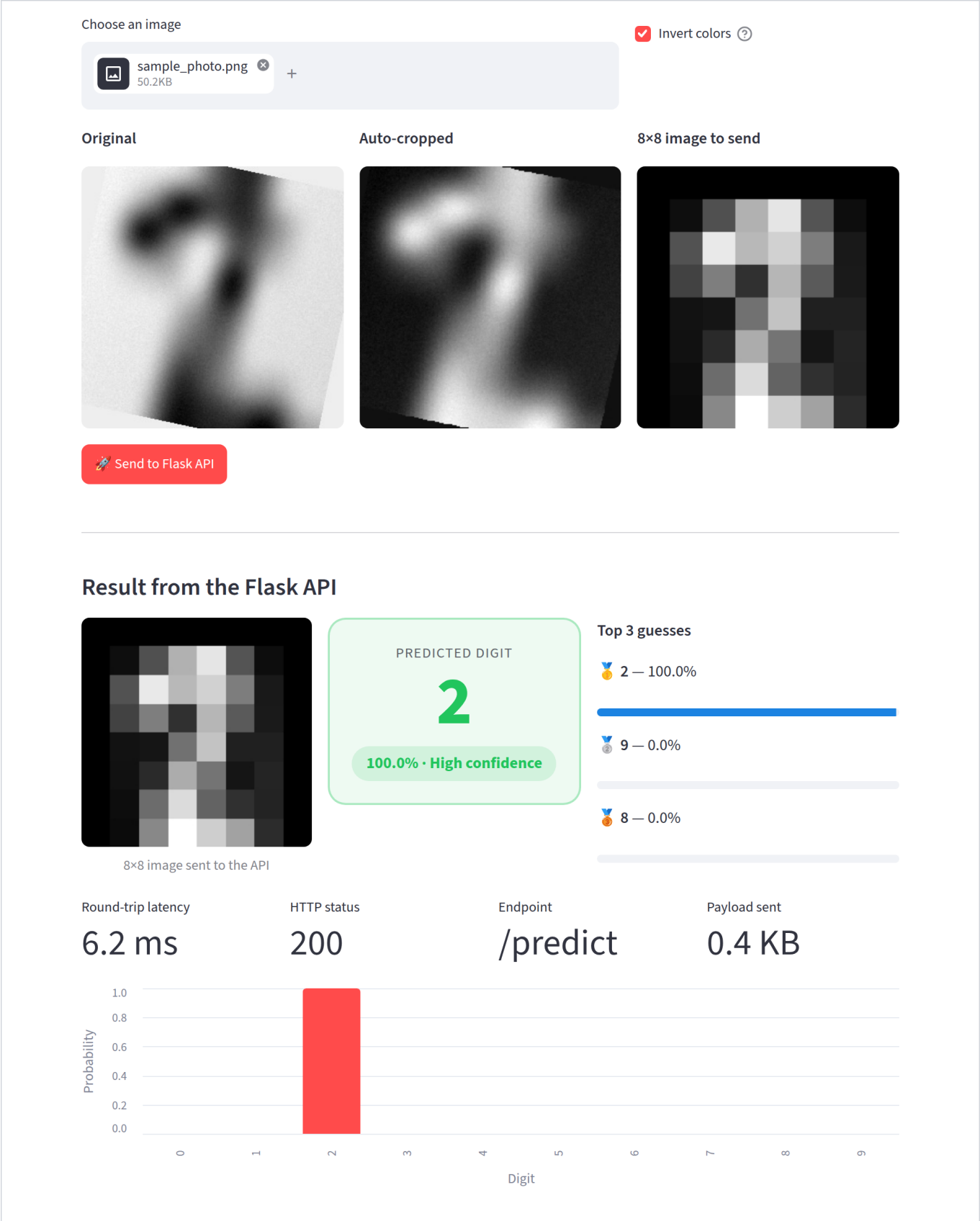


Figure 5: Uploaded image of a '2': original, auto-cropped ink and the 8x8 image that is sent, followed by the API's reply: predicted 2 with high confidence.

6.6 Batch Test on Raw Images

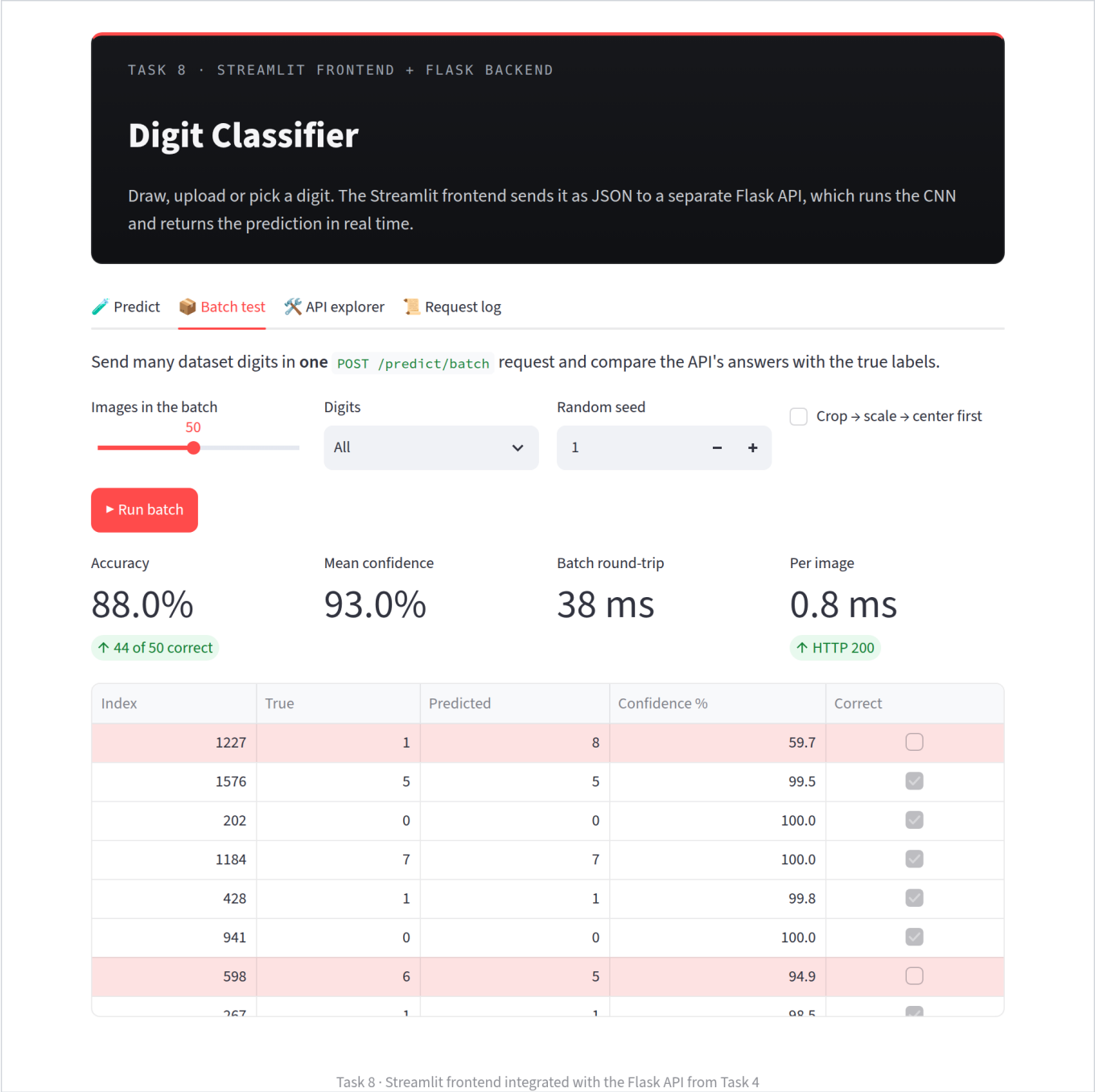


Figure 6: 50 random dataset images sent as raw 8×8 data in one POST /predict/batch request: 88.0 % accuracy (44 of 50), one round trip for the whole batch, misclassified rows highlighted.

6.7 Batch Test with Frontend Preprocessing

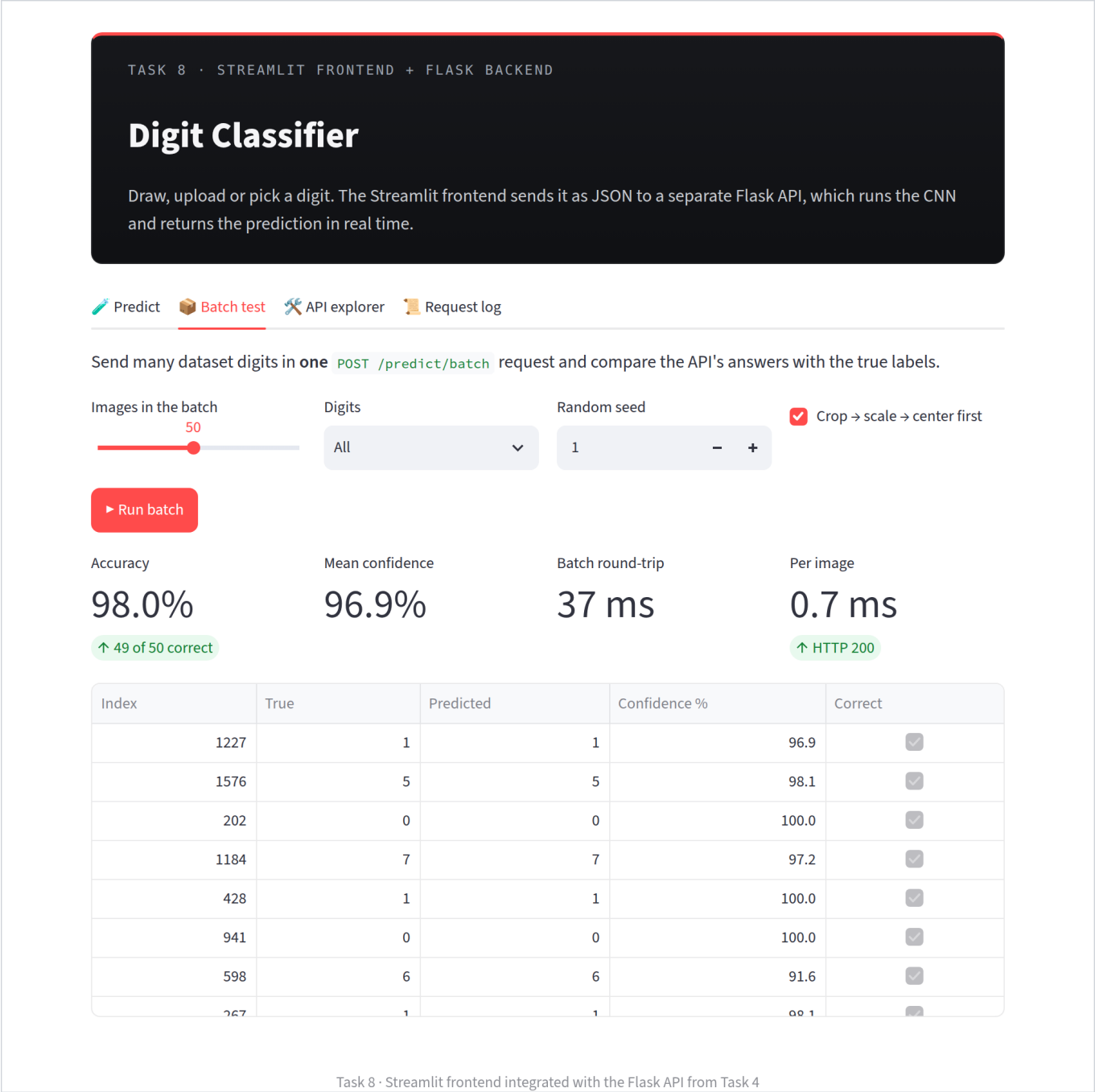


Figure 7: The same 50 images after the frontend's crop, scale and centre step: 98.0 % accuracy (49 of 50), showing why preprocessing is done before the request.

6.8 API Explorer: Handled Errors (400 and 404)

PredictBatch testAPI explorerRequest log

Send any request to the Flask API and inspect exactly what comes back — including deliberately broken ones.

Preset request

POST /predict — wrong size (3 values)

MethodPath

POST/predict

Request body

{"image": [1, 2, 3]}

Send request

HTTP 400 · 5.5 ms · handled error (structured JSON, server stays up)

{

"error" : "Expected an 8x8 (64-value) grayscale image, got 3 values."

}

PredictBatch testAPI explorerRequest log

Send any request to the Flask API and inspect exactly what comes back — including deliberately broken ones.

Preset request

GET /nonexistent (expect 404)

MethodPath

GET/nonexistent

Request body

Send request

HTTP 404 · 5.5 ms · handled error (structured JSON, server stays up)

{

"error" : "Endpoint not found. See GET / for a list of available endpoints."

}

Figure 8: Deliberately wrong requests sent from the UI. Top: an image with 3 values returns HTTP 400 with an explanatory JSON message. Bottom: an unknown endpoint returns HTTP 404 with a pointer to the available endpoints.

6.9 API Explorer: One Bad Image inside a Batch

PredictBatch testAPI explorerRequest log

Send any request to the Flask API and inspect exactly what comes back — including deliberately broken ones.

Preset request

POST /predict/batch — one bad image in the batch

Method

POST

Path

/predict/batch

Request body

{
 "images": [[0, 0, 5, 13, 9, 1, 0, 0, 0, 0, 13, 15, 10, 15, 5, 0, 0, 3, 15, 2, 0, 11, 8, 0, 0, 4, 12, 0, 0, 8, 8, 0, 0, 5, 8, 0, 0, 9, 8, 0, 0, 4, 11, 0, 1, 12, 7, 0, 0, 2, 14, 5, 10, 12, 0, 0, 0, 0, 6, 13, 10, 0, 0, 0], [1, 2, 3]]
}

Send request

HTTP 200 · 11.0 ms · success

{
 "count" : 2
 "results" : [
 0 : {
 "confidence" : 1
 "predicted_digit" : 0
 "probabilities" : {
 "0" : 1
 "1" : 0
 "2" : 0
 "3" : 0
 "4" : 0
 "5" : 0
 "6" : 0
 "7" : 0
 "8" : 0
 "9" : 0
 }
 }
 1 : {
 "error" : "image[1]: Expected an 8x8 (64-value) grayscale image, got 3 values."
 }
]
}

Task 8 · Streamlit frontend integrated with the Flask API from Task 4

Figure 9: A batch with one valid and one invalid image returns HTTP 200: the first entry is a prediction and the second an error message for image[1], so one bad image does not fail the batch.

6.10 Request Log

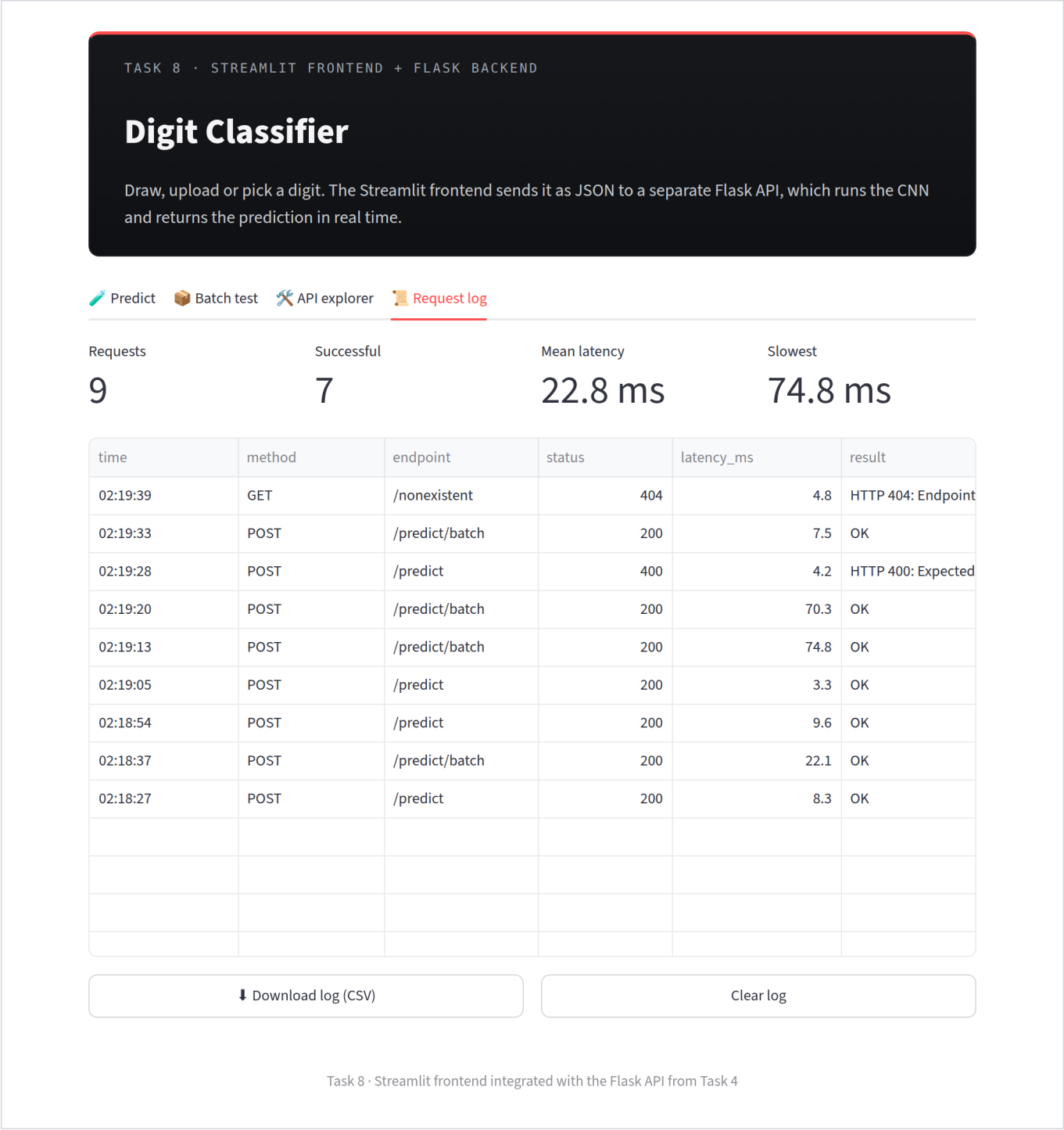


Figure 10: Every call made in the session: nine requests, seven successful. The two failures are the deliberate 400 and 404 from the API explorer. The log can be downloaded as CSV.

6.11 API Offline: Graceful Degradation

<<

Deploy ⋮

🔌 Flask API connection

API base URL

http://127.0.0.1:5000

Request timeout (s)

5

Re-check connection

API offline X

Cannot reach the Flask API at <http://127.0.0.1:5000> (connection refused).

cd flask_api

python app.py

Requests sent this session: 0

TASK 8 · STREAMLIT FRONTEND + FLASK BACKEND

Digit Classifier

Draw, upload or pick a digit. The Streamlit frontend sends it as JSON to a separate Flask API, which runs the CNN and returns the prediction in real time.

Predict

Batch test

API explorer

Request log

1 Choose how to provide a digit.

2 Press **Send to Flask API** — the frontend posts JSON and shows the live reply.

☒ Sample digit

☐ Draw

☐ Upload photo

☐ Test-time augmentation (sends 11 variants in one /predict/batch call and averages them)

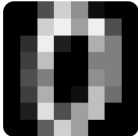
Digit

All

Sample number

1

☐ Crop → scale → ...



Dataset index 0 · true label 0

🚀 Send to Flask API

The Flask API is offline, so predictions are unavailable. Start it, then press **Re-check connection**.

Task 8 · Streamlit frontend integrated with the Flask API from Task 4

Figure 11: A fresh visit while the Flask API is stopped: the sidebar shows “API offline” with the connection error and the command to start the backend, the Send button is disabled, and a warning explains why.

7. Test Report and Results

Two automated suites were run against the real application. The integration suite (`tests/run_integration_tests.py`) starts the Flask API in a subprocess and calls it through the same client the frontend uses. The end-to-end suite (`tests/e2e_ui_tests.py`) drives the Streamlit UI in a real browser while Flask runs in the background. Evaluation images are the 270-image validation split of the Task 7 evaluation (seed-42 permutation, last 15 %) and the full 1,797-image dataset.

7.1 Summary

Test group	Tests	Passed
Connection	2	2
Prediction	7	7
Errors	11	11
Faithfulness	1	1
Accuracy	1	1
Performance	3	3
Resilience	2	2
Browser end-to-end checks	12	12
Total	39	39

7.2 API integration tests

ID	Test	Expected	Actual	Result
T0 1	GET / returns the API description	200 + 3 endpoints listed	HTTP 200, endpoints=3	PASS
T0 2	GET /health reports the model as loaded	200, model_loaded = true	HTTP 200, {'model_loaded': True, 'status': 'ok'}	PASS
T0 3	Single raw image (true 3) via POST /predict	digit 3, probabilities sum to 1	HTTP 200, digit 3 @ 0.9901, sum(p)=0.9999	PASS
T0 4	Frontend-preprocessed image (true 7) via POST /predict	digit 7	digit 7 @ 0.9989	PASS
T0 5	0-16 and 0-1 pixel scales give the same digit	both predict 0	0-16 scale -> 0, 0-1 scale -> 0	PASS
T0 6	Nested 8x8 list is accepted	200, digit 0	HTTP 200, digit 0	PASS
T0 7	Batch of 3 real digits (3, 7, 8)	[3, 7, 8]	HTTP 200, predicted [3, 7, 8], count=3	PASS
T0 8	Test-time augmentation: 11 variants in one batch call	11 results, averaged digit 8	11 variants, averaged digit 8 @ 0.5905	PASS
T0 9	Batch at the limit (100 images)	200, 100 results	HTTP 200, count=100	PASS
T1 0	Missing 'image' field	400	HTTP 400: Missing 'image' field in request body.	PASS
T1 1	Wrong-size image (3 values)	400	HTTP 400: Expected an 8x8 (64-value) grayscale image, got 3 values.	PASS

ID	Test	Expected	Actual	Result
T1 2	Invalid JSON body	400	HTTP 400: Request body must be valid JSON.	PASS
T1 3	Pixel value above range (99)	400	HTTP 400: Pixel values must be within [0, 16] (or already normalized to [0, 1]).	PASS
T1 4	Negative pixel value	400	HTTP 400: Pixel values must be within [0, 16] (or already normalized to [0, 1]).	PASS
T1 5	Non-numeric pixel values	400	HTTP 400: could not convert string to float: 'a'	PASS
T1 6	Unknown endpoint	404	HTTP 404: Endpoint not found. See GET / for a list of available endpoints.	PASS
T1 7	Wrong HTTP method (GET /predict)	405	HTTP 405: Method not allowed for this endpoint.	PASS
T1 8	Empty batch list	400	HTTP 400: 'images' must be a non-empty list of 64-value images.	PASS
T1 9	Batch larger than 100	400	HTTP 400: Batch size limited to 100 images per request.	PASS
T2 0	One bad image does not fail the whole batch	200; entries [ok, error, ok]	HTTP 200: entries -> ['ok', 'error', 'ok']	PASS
T2 1	API output equals the local model on all 1,797 images	identical predictions, diff < 1e-4	1797/1797 identical predictions, max p_api - p_local = 5.0e-05	PASS
T2 2	Accuracy measured through the HTTP API	> 90 % raw, > 95 % preprocessed (held-out)	{"held_out_270_raw": 90.74, "held_out_270_preprocessed": 98.52, "all_1797_raw": 89.76, "all_1797_preprocessed": 98.5}	PASS
T2 3	200 sequential POST /predict calls	p95 round-trip < 250 ms	median 2.0 ms, p95 2.4 ms, max 3.0 ms	PASS
T2 4	POST /predict/batch with 100 images (median of 5)	completes; per-image cost reported	100 images in 36.1 ms (0.36 ms per image)	PASS
T2 5	200 requests from 8 parallel clients	all 200 succeed	200/200 succeeded with 8 parallel clients, 411.2 requests/s	PASS
T2 6	API stopped: client reports an error instead of crashing	unreachable result with message	health -> 'Cannot reach the Flask API at http://127.0.0.1:5000 (connection refused).'	PASS
T2 7	API restarted: the same client works again	prediction succeeds	API restarted; first prediction OK after 0.41 s	PASS

7.3 Browser end-to-end checks

ID	Check	Expected	Actual	Result
U0 1	App loads and the sidebar shows the Flask API as online	'API online' in sidebar	API online ✓ · model loaded · 8 ms	PASS
U0 2	Sample digit 7: digit shown in the UI equals the API's answer	UI = API = 7	UI shows 7, API returns 7	PASS
U0 3	Test-time augmentation sends 11 variants in one /predict/batch request	endpoint '/predict/batch'	Round-trip latency 22.1 ms HTTP status 200 Endpoint /predict/batch Payload sent 3.6 KB	PASS

ID	Check	Expected	Actual	Result
U04	Hand-drawn '3' is sent to the API and the prediction is shown	digit 3	UI shows 3	PASS
U05	Uploaded photo of a '2' is preprocessed, sent and classified	digit 2	UI shows 2	PASS
U06	Batch tab: 50 images in one request; UI accuracy equals a direct API call	raw accuracy = 88.0%	UI raw 88.0% UI preprocessed 98.0% direct raw 88.0%	PASS
U07	API explorer: 'POST /predict — wrong size (3 values)' shows HTTP 400	HTTP 400	page shows 'HTTP 400'	PASS
U07	API explorer: 'POST /predict/batch — one bad image in the batch' shows HTTP 200	HTTP 200	page shows 'HTTP 200'	PASS
U07	API explorer: 'GET /nonexistent (expect 404)' shows HTTP 404	HTTP 404	page shows 'HTTP 404'	PASS
U08	Request log records every call made by the UI	log has ≥ 8 requests	9 requests logged; Requests 9 / Successful 7 / Mean latency 22.8 ms / Slowest 74.8 ms	PASS
U09	API stopped: UI shows 'API offline', explains how to start it, disables Send	offline banner + disabled button	sidebar: API offline ✗ · Cannot reach the Flask API at http://127.0.0.1:5000 (connection refused). Send button disabled = True	PASS
U10	API restarted: UI reconnects and predictions work again	'API online' + a result	sidebar: API online ✓ · model loaded · 8 ms; a new result is displayed	PASS

7.4 Accuracy measured through the HTTP API

Images	Raw 8×8 images	After frontend preprocessing
270 validation images	90.74%	98.52%
All 1,797 dataset images	89.76%	98.50%

Faithfulness check: all 1,797 images were classified both through the HTTP API and directly by the model code inside the API process. 1,797 of 1,797 predictions were identical and the largest probability difference was 5.0e-05, which is exactly the 4-decimal rounding the API applies to its JSON output. The integration therefore changes nothing about the model's answers.

7.5 Latency, throughput and recovery

Measurement	Result
Single <code>POST /predict</code> , 200 sequential calls	mean 2.0 ms · median 2.0 ms · 95th percentile 2.4 ms · 99th 2.6 ms · max 3.0 ms
<code>POST /predict/batch</code> with 100 images (median of 5)	36.1 ms in total, 0.36 ms per image
200 requests from 8 parallel clients	200 of 200 succeeded in 0.49 s (411.2 requests/s)
Flask start-up until health check passes	0.21 s
API restart until the first prediction succeeds	0.41 s

All timings are measured on one machine over the loopback interface, so they exclude real network delay, and use Flask's built-in development server.

7.6 Flask server console log during the browser test

The backend's own log of the requests that the Streamlit UI sent during the end-to-end test: 38 requests, 36 × HTTP 200, 1 × HTTP 400, 1 × HTTP 404. The single 400 and 404 are the deliberate error cases from the API explorer; the server kept answering normally afterwards.

```
127.0.0.1 - - [08/Oct/2026 02:18:09] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:22] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:26] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:27] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:27] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:30] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:36] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:37] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:37] "POST /predict/batch HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:42] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:43] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:49] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:51] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:54] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:18:54] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:01] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:02] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:05] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:05] "POST /predict HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:13] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:13] "POST /predict/batch HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:18] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:19] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:19] "POST /predict/batch HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:22] "POST /predict/batch HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:26] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:28] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:28] "POST /predict HTTP/1.1" 400 -
127.0.0.1 - - [08/Oct/2026 02:19:32] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:33] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:33] "POST /predict/batch HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:38] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:39] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:39] "GET /nonexistent HTTP/1.1" 404 -
127.0.0.1 - - [08/Oct/2026 02:19:56] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:56] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:59] "GET /health HTTP/1.1" 200 -
127.0.0.1 - - [08/Oct/2026 02:19:59] "POST /predict HTTP/1.1" 200 -
```

8. Observations

- **The integration works end to end.** All 27 API tests and all 12 browser checks pass. The digit shown in the browser always equals the digit returned by a direct API call (U02, U06), and the API's answers equal the local model's on all 1,797 images.
- **Predictions are real-time.** A single request takes a median of 2.0 ms and 95 % finish within 2.4 ms; a batch of 100 images takes 36.1 ms (0.36 ms per image). What the user waits for is the Streamlit page update, not the inference.
- **Preprocessing is essential.** Through the same API, raw dataset images score 90.74% on the 270 validation images but 98.52% after the frontend's crop, scale and centre step (and 89.76% against 98.50% on all 1,797 images). The model was trained on preprocessed images, so a client that skips this step loses about eight points. The UI batch demonstration shows the same effect (88.0% against 98.0% on 50 images).
- **Test-time augmentation tempers confidence.** For the sample '7', one request gives 99.4 % confidence, while the average over 11 variants gives 67.4 % for the same digit: the variants disagree a little, and averaging reflects that uncertainty.
- **Errors are handled at both ends.** The API answers invalid input with structured JSON (400, 404, 405) and keeps running; the frontend shows these messages, surfaces a partial failure inside a batch, and never shows a stack trace.
- **The system is reliable under failure and load.** With the API stopped, the client returns a readable error instead of raising (T26) and the UI shows an offline state, disables Send and explains how to start the backend (U09). After a restart (0.41 s) the same client and the same page work again (T27, U10). Eight parallel clients sent 200 requests with no failures.
- **Testing improved the application.** The browser test found a real crash in the drawing tab and several presentation problems (Section 4.5), all fixed before the final run.

9. Known Limitations

- **Model weights.** The backend serves the frozen model from the Task 7 application because the Task 1 weights file was not supplied; Task 4's logged confidences therefore differ slightly from this model's. The code is weights-agnostic.
- **Validation split.** The 270 “validation” images are rebuilt from a seed-42 permutation, as in Task 7. The training script is not part of this submission, so the split cannot be proved disjoint from the training data; the full-dataset figures are given alongside for that reason.

- **Localhost measurements.** Latency figures exclude network delay and use Flask's development server. A production deployment would use a WSGI server such as gunicorn and would add HTTPS and authentication, none of which are implemented.
- **Server-side requests.** The API address must be reachable from the machine running Streamlit. If the API is moved, change the URL in the sidebar or set `DIGIT_API_URL`.
- **Small, low-resolution data.** The model works on 8×8 images; very tall, wide or ornate digits stay ambiguous (see the Task 7 report).
- **Synthetic upload test.** The photo used for the upload test was generated from a dataset digit, not photographed; real photographs may behave differently.
- **Batch limit.** The API accepts at most 100 images per batch request, and the frontend sends test-time augmentation and batch tests within that limit.

10. Deliverables Produced

Deliverable	File
Integrated application — Flask API (Task 4, unchanged)	<code>flask_api/app.py</code>
Integrated application — Streamlit frontend	<code>frontend/streamlit_app.py</code> , <code>frontend/api_client.py</code> , <code>frontend/preprocessing.py</code>
Model served by the API	<code>flask_api/cnn_digits_model.pkl</code>
Workflow documentation	Section 4 of this report and <code>README.md</code>
Test report	Section 7 of this report; <code>tests/results/test_results.json</code> , <code>tests/results/ui_test_results.json</code>
Automated tests	<code>tests/run_integration_tests.py</code> , <code>tests/e2e_ui_tests.py</code>
Screenshots and server logs	<code>docs/screenshots/</code> , <code>tests/results/e2e_flask_server.log</code>
Dependency list	<code>requirements.txt</code>

Evaluation criterion	Evidence
Successful integration	UI result equals a direct API call (U02, U06); API equals the local model on 1,797 of 1,797 images; Flask log shows the UI's requests (Section 7.6).
Real-time prediction capability	Median round trip 2.0 ms, 95th percentile 2.4 ms; 100-image batch in 36.1 ms (Section 7.5).
Application reliability	All 27 API tests and 12 browser checks pass; clean JSON errors for 10 kinds of bad input (T10–T19); graceful offline state and recovery in 0.41 s; 200 concurrent-client requests without failure.

11. Conclusion

The Task 4 Flask API was connected to a new Streamlit frontend through a small, fully error-handled HTTP client. A user can pick, draw or upload a digit and see the model's prediction, confidence, probabilities and round-trip time within milliseconds, while the model stays entirely on the backend. The workflow was validated at three levels: 27 API integration tests, a 12-check browser end-to-end test, and a faithfulness check showing the API reproduces the model's output on all 1,797 images. The testing also made one design lesson concrete: because the model expects preprocessed digits, the frontend must normalise hand-drawn and photographed input, which raises accuracy through the API from 90.7% to 98.5% on the validation images. The report states its limits plainly: localhost timings, a development server, the substituted weights file, and no authentication.